EcoMesh: A Distributed Peer-to-Peer Disaster Relief Communication Mesh

Mini Project Report

submitted in partial fulfillment of the requirement
for the award of the Degree of

Bachelor of Technology
in
Computer Engineering

by

Miti Shah	**UID 2024300230**
Twissha Shah	**UID 2024300231**
Tanvi Sheth	**UID 2024300239**

T.E. Semester V • Mini Project – I • Group 20

under the guidance of

Prof. Anuj Tawari



Department of Computer Engineering

Bharatiya Vidya Bhavan's
Sardar Patel Institute of Technology
(Autonomous Institute Affiliated to University of Mumbai)
Munshi Nagar, Andheri (West), Mumbai – 400058

Academic Year 2026–27

Approval Certificate

This is to certify that the Mini Project entitled **"EcoMesh: A Distributed Peer-to-Peer Disaster Relief Communication Mesh"** by **Miti Shah**, **Twissha Shah** and **Tanvi Sheth** is approved for the partial fulfillment of Mini Project – I for the Third Year Mini Project towards obtaining the Degree of Bachelor of Technology in Computer Engineering.

Project Guide

_____
(Signature)

Name: _____

Date: _____

External Examiner

_____
(Signature)

Name: _____

Date: _____

_____
Head of Department

_____
Principal

Seal of the Institute

Declaration

We wish to state that the work embodied in this report titled "EcoMesh: A Distributed Peer-to-Peer Disaster Relief Communication Mesh" forms our own contribution to the work carried out under the guidance of Prof. Anuj Tawari at the Sardar Patel Institute of Technology. We declare that this written submission represents our ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. The report distinguishes the features that are implemented and tested from those that are proposed, and every reported result is supported by stored evidence.

Miti Shah	**Twissha Shah**	**Tanvi Sheth**
UID 2024300230	UID 2024300231	UID 2024300239

Date: _____

Place: Mumbai

Acknowledgement

We would like to express our sincere gratitude to our project guide, Prof. Anuj Tawari, for the direction, the regular reviews and the candid feedback that shaped this project from a problem statement into a working, measured system. His insistence that every claim be backed by evidence is reflected throughout this report.

We thank the Head of the Department of Computer Engineering and the Principal of the Sardar Patel Institute of Technology for providing the environment, the laboratory facilities and the opportunity to carry out this mini project. We are grateful to the faculty who coordinated the evaluation phases and the report guidelines, and to the examiners whose comments at the earlier phases improved the design.

Finally, we thank our classmates, who tested the demonstration and gave honest feedback on the interface, and our families for their support throughout the semester.

Miti Shah
Twissha Shah
Tanvi Sheth

Abstract

Problem. An earthquake, fire or collapse can take down the towers and internet links that every popular messaging application needs, although the phones of the people trapped inside can often still reach one another. Whatever connectivity survives helps only the person who holds it.

Motivation and objectives. Rescue is time-critical, so the aim is to let one person's connectivity serve everyone nearby and to let an SOS overtake routine traffic at every hop: a self-organising mesh, routing weighted by measured link quality, reliable delivery with a hold-and-forward fallback, encrypted and signed messages, and measured performance.

Proposed solution. In EcoMesh every device both sends and relays. A message moves device by device until it reaches a gateway with an outside link or a rescue device, or is held until a path appears. No internet, tower or server is needed.

Technologies and engineering concepts. Python `asyncio` (one event-loop thread per node); a 52-byte binary header of our own design; link-state gossip; Dijkstra over the cost $1+5(1-q)+3f$; a priority queue at every hop; ACK with retry and re-routing; AES-256-GCM and Ed25519; a SQLite log per node; FastAPI and WebSocket; React; and a transport interface that runs one engine on a simulator, on loopback and across laptops.

Key implementation features. Discovery with measured link quality; link-state flooding controlled by sequence numbers; duplicate suppression; a bounded hold queue that never drops an SOS for routine traffic; signatures that survive relaying; and one device app that becomes a survivor screen (hold-to-send SOS with nine request types, location, voice input, "I am safe", the rescue reply), a gateway view or a rescue console (requests with their route, replies, filters, history, CSV export, live map and activity panel).

Testing methodology. 334 automated tests (unit, integration, boundary, fuzz, replay, concurrency, a three-node mesh over TCP), a test mapping for all 68 requirements, seven seeded simulator scenarios, loss and scale sweeps to 65 nodes, five live processes over real sockets, and scripted browser checks of the device screens (21 of 21 pass).

Results. A 40-node mesh at 20 % loss with injected failures delivered 92.6 % of messages (50 runs). On identical seeds weighted routing delivered 99.2 % against 57.4 % for hop count. Under load an SOS had a 4.4× lower median latency than routine traffic. On five live processes an SOS arrived in a median 32 ms, a sender re-routed around a failed relay in a median 4.1 s, and every message held during a gateway outage was delivered. Convergence and recovery met their targets on average but not in every run; at 20 % loss delivery stayed above 90 % to 50 nodes and fell to 83.1 % at 65.

Practical applicability and limitations. EcoMesh can serve today as an institutional safety network on laptops. The radio is emulated by a LAN restricted to configured neighbours, so it is not yet phone to phone; a Bluetooth or Wi-Fi Direct transport, adaptive timers, a usability study and per-pair keys are the main future work.

Table of Contents

Abstract v

Table of Contents vi

List of Figures vii

List of Tables vii

List of Abbreviations and Symbols viii

1 Introduction and Problem Definition 1

2 Literature Review and Study of Existing Systems 3

3 Software Requirements Specification 5

4 System Architecture and Design 7

5 Algorithm Design and Data Structures 13

6 Subject-Wise Engineering Concepts and APNI Leap Alignment 16

7 Implementation and Module-Wise Coding 17

8 Scalability, Performance and Resource Management 20

9 Testing and Validation 23

10 Results, Demonstration and Discussion 26

11 Conclusion and Future Scope 30

References 31

Appendix: Research Paper / Poster and Plagiarism Report 32

List of Figures

Figure 1 Block diagram: one EcoMesh node and its three interchangeable transports . . . 7
Figure 2 Level-0 (context) data flow diagram . . . 8
Figure 3 Level-1 data flow diagram of one node . . . 9
Figure 4 Use case diagram . . . 9
Figure 5 Class diagram: `Node` as the composition root . . . 10
Figure 6 Activity diagram: the packet receive path . . . 10
Figure 7 Sequence diagram: an SOS relayed hop by hop . . . 11
Figure 8 State machine of the message lifecycle and deployment diagram . . . 11
Figure 9 Entity–relationship diagram of the per-node database . . . 12
Figure 10 Flowchart: originate, acknowledge, retry and retain a message . . . 15
Figure 11 Flowchart: next-hop selection, including `RESCUE__` . . . 15
Figure 12 Delivery, routing comparison, scale and recovery results . . . 21
Figure 13 The request form and the gateway view . . . 27
Figure 14 The survivor's screen: delivered, held and flushed . . . 28
Figure 15 The rescue console, before and during a gateway outage . . . 29
Figure 16 The live panel of what the device is doing . . . 29

List of Tables

Table 1 Key performance indicators . . . 2
Table 2 Twelve studies: technique, contribution and relevance to EcoMesh . . . 3
Table 3 Existing systems and routing protocols compared with EcoMesh . . . 4
Table 4 Functional requirements by feature . . . 6
Table 5 The 52-byte packet header . . . 8
Table 6 Sample rows of `messages` for one SOS from A to C in a real run: each node holds its own view of the same message id . . . 12
Table 7 Threat model and countermeasures . . . 12
Table 8 Data structures and why they were chosen . . . 14
Table 9 Time complexity of the critical operations . . . 15
Table 10 Subjects, concepts applied, modules and evidence . . . 16
Table 11 Modules: code reference, logic, dependencies and tests . . . 18
Table 12 Commits per team member, read from `git log` over all remote branches (98 commits) . . . 19
Table 13 Results of the seven simulator scenarios . . . 21
Table 14 Scalability, 10 to 65 simulated nodes . . . 22
Table 15 Live devices: 5 operating-system processes, real TCP sockets, real timers, on one laptop . . . 22
Table 16 Browser checks of the device screens (the one app), run against five live device processes . . . 24
Table 17 Representative test cases . . . 24
Table 18 Significant defects, causes, corrective actions and retesting . . . 25
Table 19 Validation of the key performance indicators of Table 1 against measured data . . . 25
Table 20 Feature status . . . 26
Table 21 Objectives, supporting evidence and status . . . 27

List of Abbreviations and Symbols

Term	Meaning
ACK	Acknowledgement packet returned by the destination along the reverse path
AES-256-GCM	Advanced Encryption Standard, 256-bit key, Galois/Counter Mode (authenticated encryption)
AODV	Ad hoc On-Demand Distance Vector routing (RFC 3561)
API	Application Programming Interface
ASGI	Asynchronous Server Gateway Interface (used by FastAPI/uvicorn)
BLE	Bluetooth Low Energy
DAA	Design and Analysis of Algorithms
DBMS	Database Management System
DFD	Data Flow Diagram
DTN	Delay-Tolerant Networking (RFC 4838)
Ed25519	Edwards-curve Digital Signature Algorithm over Curve25519
ER	Entity–Relationship (diagram)
ESE	End Semester Evaluation
FR / NFR	Functional / Non-Functional Requirement
GATEWAY	Node role: has a link to the outside world; valid terminal for `RESCUE__`
HELLO	Periodic one-hop beacon used for neighbour discovery
KPI	Key Performance Indicator
LAN	Local Area Network
MANET	Mobile Ad hoc Network
OLSR	Optimized Link State Routing (RFC 3626)
OS	Operating System
PWA	Progressive Web App
RESCUE	Node role: rescue-team device; valid terminal for `RESCUE__`
`RESCUE__`	Symbolic destination meaning “the nearest GATEWAY or RESCUE node”
SF / S&F	Store-and-Forward
SOS	Highest message priority (value 0)
SRS	Software Requirements Specification
TCP / UDP	Transmission Control Protocol / User Datagram Protocol
TTL	Time To Live (remaining hop allowance of a packet)
UML	Unified Modeling Language
WAL	Write-Ahead Logging (SQLite journal mode)
V, E	Number of vertices (nodes) and edges (links) of the mesh graph
q	Link quality in $[0, 1]$: fraction of the last 20 expected HELLOs received
f	Recent failures: ACK timeouts attributed to a link in the last 60 s
C_{max}, T_{hold}	Store-and-forward bounds: maximum count (500) and maximum age (1800 s)

CHAPTER 1

Introduction and Problem Definition

1.1 Project Background

After an earthquake, fire or collapse the mobile network and internet link are usually the first to fail or saturate, yet connectivity survives in pockets (a person near a window with one bar of signal, an access point on a UPS, a rescue team's kit) that help only their owner. **EcoMesh** turns every participating device into both a sender and a relay, so a message can travel from device to device until it reaches its addressee, a *gateway* that still has an outside link, or a *rescue* device, without the internet and without a server.

1.2 Problem Statement

WhatsApp, Signal, Telegram and SMS all need a distant server or tower; when a Wi-Fi router loses its uplink they all stop, although the phones on it can still exchange packets. Trapped people then cannot report their condition, share the one weak signal, coordinate or be located quickly. Existing offline systems either flood messages or route by hop count, give no priority to emergencies, publish few measurements, and in one widely deployed case were shown to lack effective confidentiality and authenticity [1] (Chapter 2).

1.3 Motivation

Survival depends on how soon rescuers learn who is inside, where, and in what condition. Sharing one person's connectivity with everyone nearby and pushing an SOS ahead of routine traffic at every hop shortens that delay for trapped occupants, rescue teams, and the hospitals, campuses, hostels, factories and stadiums that could install it in advance.

1.4 Objectives

O1 *Self-organising mesh*: neighbours are found automatically and every node learns the whole graph within 15 s. O2 *Weighted routing* that outperforms hop count on identical scenarios. O3 *Priority*: an SOS overtakes routine traffic at every hop, with a median latency at least 3× lower under load. O4 *Reliability*: at least 90 % delivery on 40 nodes at 20 % loss, and a new route within 10 s of a relay failure. O5 *Store-and-forward*: a message without a route is kept, never dropped, and is delivered when a route returns. O6 *Security*: payloads stay confidential and tamper-evident, and a forged SOS is rejected. O7 *Evidence*: every metric is computed from the nodes' own logs. O8 *Integration*: a web front end, a REST and WebSocket back end and the engine running together on several devices.

1.5 Scope of the Project

Included: discovery, a binary wire protocol, link-state gossip, Dijkstra routing over a link-quality cost, per-hop priority, TTL, duplicate suppression, acknowledgement with retry, heartbeats, node roles, bounded store-and-forward, AES-256-GCM and Ed25519, a SQLite log per node, `asyncio` concurrency, a simulator with metrics, and survivor and rescue screens. **Excluded:** a real Bluetooth LE or Wi-Fi Direct transport, NAT traversal, voice calls, video and file transfer, machine learning and a native mobile app. **Users:** the trapped person, the relay user, the gateway operator and the rescue coordinator. **Environment and scale:** laptops running Python 3.10 on a shared network, five live devices and up to 65 simulated nodes. One design decision is stated plainly: the mesh runs over LAN sockets that stand in for Bluetooth LE or Wi-Fi Direct, and each node may talk only to the neighbours listed for it, which imitates limited radio range. Discovery, routing, reliability and encryption sit above the transport, so a radio transport can replace the sockets without changing them.

1.6 Proposed Solution

Each device runs one node, a Python process whose `asyncio` loop handles discovery, gossip, receiving, sending, store-and-forward and a small web server. A message leaves by one of three exits: a **gateway** with an outside link (reaching any gateway counts as delivery); a **rescue** device that comes into range and receives every pending SOS; or **store-and-forward**, where a node with no route keeps the message, so a person walking out carries others' messages out. Paths are chosen by Dijkstra's algorithm on a graph whose edge weights penalise weak and failing links (Chapter 5).

1.7 Novelty and Originality

EcoMesh combines **priority-aware, link-quality-weighted routing with a delay-tolerant fallback, and evaluates it by measurement**. SOS priority is applied again at every hop. The link cost is derived from measured HELLO reception and from acknowledgement failures, and a failure raises the cost immediately. A single transport interface lets the same engine run on a simulator, on loopback and on separate laptops. Every reported number is a query over the nodes' logs, and weighted and hop-count routing are compared on the same seeds.

1.8 Feasibility Analysis

Technical: all components are mature open-source software, and the largest risk, debugging routing over live sockets, was removed by building the simulator first. **Economic:** the system uses existing laptops and free software and needs no server, so there is no running cost. **Operational:** one command starts a node, and the survivor screen has one prominent control. **Time and resources:** three members over one semester, about 336 person-hours, delivered in four tagged phases.

1.9 Social Relevance and Practical Applications

The realistic path is *installation before an incident*: an institution puts EcoMesh on staff and resident devices as part of its safety plan and rescue teams carry it as standard kit, so the rescue exit exists even if no civilian has it. Survivors are found faster at no infrastructure cost.

1.10 Success Criteria and Key Performance Indicators

Table 1 lists the quantified requirements of the SRS, fixed before testing began; Section 9.10 checks each against measured data.

Table 1: Key performance indicators

KPI	Definition	Target	Measured by
K1 Delivery	Delivered ÷ sent, 40 nodes, 20 % loss	$\geq 90\,\%$	seeded simulator runs
K2 Convergence	Every node's graph holds every node	$\leq 15\,\mathrm{s}$	simulator, live
K3 Recovery	Relay death until a route avoiding it	$\leq 10\,\mathrm{s}$	50 trials, live
K4 SOS advantage	Median routine ÷ median SOS latency under load	$\geq 3\times$	burst scenario
K5 Store-and-forward	Retained messages flushed on return	100 %	simulator, live
K6 Weighted vs hop count	Delivery on identical seeds	weighted better	paired runs
K7 Scale	Nodes within the targets	≥ 50	sweep 10–65 nodes
K8 UI freshness	Event to screen	$\leq 2\,\mathrm{s}$	API time, poll interval
K9 Correctness	Tests passed	100 %	repeated runs
K10 Security	No relay decryption; forged SOS and tampering rejected	all hold	automated tests

CHAPTER 2

Literature Review and Study of Existing Systems

2.1 Study of Existing Systems

Offline messengers. *Bridgefy*, a Bluetooth messenger used at large protests, is Study 11 of Table 2; *FireChat* offered phone-to-phone chat and was withdrawn; *Briar* needs no server and exchanges encrypted messages over Bluetooth or Wi-Fi when devices meet [16]. **Dedicated radios.** *goTenna Mesh* pairs phones with radio units; *Meshtastic* uses LoRa with a controlled flood in which a node stays silent if a neighbour already repeats the packet [17]. **Research and humanitarian work.** The *Serval Project* builds disaster phone meshes with its *Rhizome* store-and-forward layer [18]; the *delay-tolerant networking (DTN)* architecture keeps data and carries it across intermittent links [10, 11]. "n/d" marks a property the sources do not document.

2.2 Review of Research Papers and Technical Resources

Table 2 summarises the twelve sources we studied, what each contributes and what it means for EcoMesh.

Table 2: Twelve studies: technique, contribution and relevance to EcoMesh

#	Source	Technique and contribution	Relevance and limitation
1	DSDV, Perkins and Bhagwat 1994 [2]	Each node keeps a table of distances in advance; numbered updates stop routing loops	Routes exist before the first SOS; counts hops only, and update traffic rises as nodes move
2	DSR, RFC 4728 [3]	Routes are found on demand and the whole path is written into each packet	A carried path allows loop checks and the "relayed via n" display; nothing is held
3	AODV, RFC 3561 [4]	A flooded request finds a destination and the reply sets up the route backwards	Counts hops; no holding of messages and no priority
4	OLSR, RFC 3626 [5]	Link state maintained in advance; selected relays reduce repeated broadcasts	Nearest relative of EcoMesh; still uses hop count by default
5	OSPF v2, RFC 2328 [6]	Numbered link-state flooding with Dijkstra run on each router	EcoMesh keeps this structure but uses a measured cost; designed for wired links
6	Babel, RFC 8966 [7]	Loop-free distance vector for wireless meshes with any metric plugged in	Shows the wireless metric must be replaceable; covers routing only
7	ETX, De Couto et al. 2003 [8]	Path cost from measured delivery ratios; shows that fewest-hop paths choose lossy links	Main basis for our cost; we add a penalty for acknowledgement failures
8	Epidemic routing, Vahdat and Becker 2000 [9]	Copy every message to every node encountered	Delivers well but fills buffers; we use one bounded queue instead
9	DTN architecture, Fall 2003; RFC 4838 [10, 11]	Nodes keep, carry and later pass on data, with custody handed over	Basis of our third exit; ignores priority and link quality
10	Disaster-response networks, Reina et al. 2015 [12]	Survey of ad hoc, vehicular, delay-tolerant and sensor networks in disasters	Supports our framing and the call for systems that are evaluated
11	Breaking Bridgefy, Albrecht et al. 2021 [1]	Attacks on a real mesh messenger: users tracked, messages unauthenticated, network shut down	Defines our threat model; motivates encryption and signatures

#	Source	Technique and contribution	Relevance and limitation
12	AES-GCM, Ed25519, Dijkstra [13–15]	Authenticated encryption, fast secure signatures and shortest paths	The primitives and routing algorithm we use

2.3 Comparative Analysis

Table 3 sets the systems of Section 2.1 and the protocols of Table 2 against EcoMesh: the infrastructure each needs, how it routes, whether it prioritises, whether it keeps a message that has no route, and how it protects content.

Table 3: Existing offline-communication systems and routing protocols compared with EcoMesh (n/d = not documented in the sources consulted)

System	Infrastructure	Routing	Priority	Store-fwd	Encryption
Bridgefy	Bluetooth LE phones	n/d	n/d	n/d	none effective in analysed version [1]
Briar	none (Bluetooth/Wi-Fi, Tor)	direct device sync	n/d	on contact	end-to-end [16]
Meshtastic	LoRa radios	managed flood [17]	n/d	n/d	channel-level
Serval	Wi-Fi ad hoc phones	B.A.T.M.A.N., later own	n/d	yes (Rhizome) [18]	n/d
DTN	none	architecture only	bundle class of service	yes (custody)	separate specification
AODV, DSR, DSDV, OLSR, OSPF, Babel	none	hop count, configured or pluggable metric	none	none or short	not in protocol
EcoMesh	**none**	**link state, weighted Dijkstra**	**3 levels, every hop**	**yes, bounded**	**AES-GCM + Ed25519**

2.4 Limitations of Existing Systems

Hop count, the usual default metric, picks lossy links [8]; no system prioritises an emergency at each hop; ad hoc routing drops a message with no route, while delay-tolerant designs keep it but ignore link quality and priority; and deployed messengers publish little measurement, one with no effective confidentiality or authenticity [1].

2.5 Identified Research or Engineering Gap

None of the surveyed systems joins priority-aware, link-quality-weighted routing with a delay-tolerant fallback, and none is evaluated quantitatively under controlled failures.

2.6 Proposed Improvements

EcoMesh routes over the cost $1 + 5(1 - q) + 3f$ (Section 5.2), re-applies SOS priority at each hop, holds unroutable messages in a bounded queue that never evicts an SOS for routine traffic, uses vetted cryptography, and measures all of it, including weighted against hop-count routing.

CHAPTER 3

Software Requirements Specification

3.1 Purpose and Scope of the SRS

This chapter condenses the IEEE 830 SRS of Phase II and its final revision, SRS version 2 (`docs/`), which adds the final device screens: *what* EcoMesh must do, not how. Its scope is one deployment of cooperating nodes with ten features (discovery, composition, forwarding, routing, priority, reliability, store-and-forward, security, logging, rescue dashboard) and no server.

3.2 Intended Users and Stakeholders

The *trapped user* raises an SOS and needs one clear control and no set-up; the *relay user* carries other people's traffic and must not be able to read it; the *gateway operator* holds the outside link; the *rescue coordinator* watches the SOS queue, the topology and the statistics; the *administrator* assigns node identifiers and decides which nodes can hear each other.

3.3 Overall Product Description

Every node runs the same program: HELLO beacons fill a neighbour table, link-state reports build a graph, Dijkstra picks next hops, a per-hop priority queue orders sending, ACK, retry, dedup and store-and-forward give reliability, AES-256-GCM and Ed25519 give confidentiality and authenticity, and a built-in web server shows the node in a browser. A configured peer list replaces multicast where needed (DIS-7).

3.4 Functional Requirements

The 68 functional requirements fall under the ten features; each has an identifier (DIS-1, ...) and an acceptance criterion (the tests that must pass and, for screens, a manual check). Table 4 summarises them; the SRS lists each one and `evidence/traceability.py` names its tests. The final revision adds the nine request types and voice input (MSG-1, MSG-3).

3.5 Non-Functional Requirements

Performance: at least 90 % delivery on 40 nodes at 20 % loss; convergence within 15 s; delivery resumed within 10 s of a node failure; an SOS measurably faster under load. **Scalability:** at least 50 nodes with no change of configuration. **Availability and reliability:** no single point of failure; no message discarded while there is room to keep it. **Security and privacy:** relays cannot read payloads; tampering and forged SOS are rejected; every field is validated; private keys never leave the device. **Usability:** one prominent SOS control that needs a sustained press; status on screen within 2 s; typing optional where the browser supports voice input. **Maintainability and portability:** one module per concern, and an engine independent of the link technology. **Resources:** one process on one event-loop thread with bounded queues (Chapter 8).

3.6 Hardware and Software Requirements

Hardware: laptops running Windows 11 on a Wi-Fi network or a phone hotspot; optionally a phone that opens a device's screen; no server. **Software:** Python 3.10 with `asyncio`, the `cryptography` library and SQLite in WAL mode; FastAPI, uvicorn and `websockets`; React 18 with Vite, Tailwind and d3-force; a Chromium-based browser for voice input; pytest and pytest-asyncio; Git and GitHub.

3.7 Constraints and Assumptions

Constraints: one semester (17 August to 2 November 2026); three members; no radio hardware, so a LAN restricted to configured neighbours emulates the radio; about 50 nodes; no machine learning or native app.

Table 4: Functional requirements by feature: input, processing, output and verification (Auto = automated tests; UI = automated tests plus a manual check of the screen; Insp. = code inspection; Man. = manual check)

Feature (IDs)	**Input → processing → output**	**Verification**
Peer discovery (DIS-1–7)	HELLO beacons and silences → update the neighbour table, estimate link quality over 20 HELLOs, remove silent peers → peer-up and peer-down events, link quality	7 Auto
Message composition (MSG-1–7)	a request type, location and text (typed or spoken), or a held SOS control → packet with a unique id and timestamp, encrypted, signed, queued → DATA sent; delivery state shown	3 Auto, 4 UI
Multi-hop forwarding (FWD-1–7)	DATA or ACK from a neighbour → validate, drop duplicates, check TTL and loops, append own id, queue by priority → forwarded, or dropped with a logged reason	7 Auto
Adaptive routing (RTE-1–8)	link-state reports, a destination → merge into the graph, Dijkstra over the weighted cost, resolve `RESCUE__` → least-cost next hop, recomputed on change	8 Auto
Priority handling (PRI-1–6)	packets of mixed priority → queue by priority then age at every hop → SOS sent before routine traffic	4 Auto, 1 UI, 1 Man.
Reliability (REL-1–8)	delivered or unacknowledged packets, duplicates, missing beacons → ACK, timer, retry on a new route, dedup, failure detection → bounded retries, no duplicates	8 Auto
Store-and-forward (SNF-1–6)	a message with no route → retention bounded by count and age, flush on a timer and when a neighbour appears, lowest priority evicted first → delivered when a path appears, or expired and logged	5 Auto, 1 Man.
Message security (SEC-1–7)	payload and identity fields → AES-256-GCM, Ed25519 signature, checks on receipt → tampered or forged packets rejected	6 Auto, 1 Insp.
Logging and history (LOG-1–6)	every message and event → written to the node's SQLite database → filtered history, CSV export and statistics by query	4 Auto, 1 UI, 1 Insp.
Rescue dashboard (DSH-1–6)	node state and events → topology, SOS queue and counters from the graph and logs → live map; requests by priority then age	6 UI
Total (68)		**52 Auto, 12 UI, 2 Insp., 2 Man.**

Assumptions: the software is installed before an incident; every device can reach another directly or through others; batteries last the incident; a gateway or rescue node exists for messages to leave by; node identifiers are unique.

3.8 Use Cases and User Stories

UC1: a trapped user picks a request type, gives a location by typing or speaking, and holds the SOS control; the message is sent at the highest priority and its progress is shown. **UC2:** with no route the message is held and sent when a path appears. **UC3:** a relay user forwards traffic automatically without reading it. **UC4:** a gateway node announces its role. **UC5:** a rescue coordinator sees the requests with their routes, replies and filters them. **UC6:** an administrator sets which nodes can hear each other in a topology file.

CHAPTER 4

System Architecture and Design

4.1 Overall System Architecture

Each node is a single operating-system process. Its `asyncio` event loop runs, on one thread, discovery (a HELLO every 3 s, a 10 s timeout), link-state gossip (every 5 s), the receive path, the priority-ordered sender, the store-and-forward flush (every 10 s) and the API server. All of them reach the network only through the `Transport` interface (Figure 1). Because the shared state (neighbour table, mesh graph, map of packets awaiting an ACK) is touched by one thread only, no locks are needed and data races cannot occur.

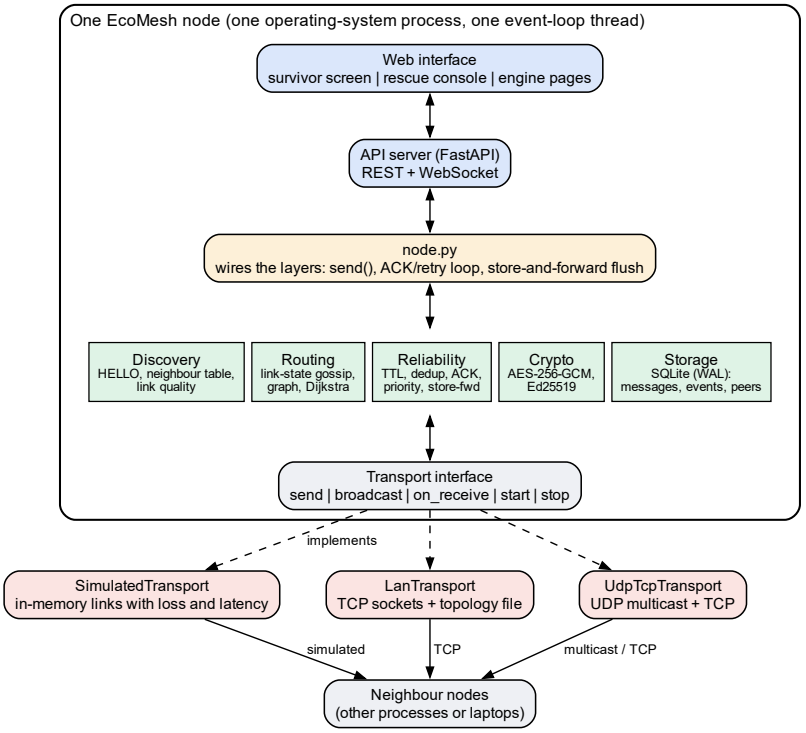


Figure 1: Block diagram: one EcoMesh node and its three interchangeable transports

4.2 Component and Module Description

The engine has one module per concern: `packet.py` (wire format), the transports, `discovery.py`, `routing.py`, `relay.py`, `reliability.py`, `crypto.py` and `store.py`, joined only in `node.py`. `sim/` and `api/` drive the engine for experiments, and `story/` serves the device screens. Table 11 in Section 7.3 gives each module's input and output, logic, dependencies and tests.

Wire protocol. A fixed big-endian header (Table 5) is followed by the path travelled so far, the payload and an optional 64-byte signature. The path is carried so that loops can be detected and the screens can show the route. HELLO is a one-hop broadcast, LINK_STATE is sent to each neighbour, and DATA and ACK travel over TCP on each hop, so every hop is ordered and reliable even though the end-to-end path is not.

Table 5: The 52-byte packet header

Off.	Size	Field	Meaning	Off.	Size	Field	Meaning
0	2	magic	`0x4D45` “ME”	24	8	src	node id, ASCII
2	1	version	1	32	8	dst	node id / `RESCUE__`
3	1	type	HELLO, LINK_STATE, DATA, ACK	40	8	timestamp	Unix ms
4	1	priority	0 SOS, 1 NORMAL, 2 STATUS	48	2	payload_len	bytes
5	1	ttl	per-hop decrement (10)	50	1	path_len	ids in path
6	1	flags	enc., needs_ack, s-f, signed	51	1	attempt	retry counter
7	1	hop_count	per-hop increment	8	16	msg_id	UUIDv4

4.3 Architecture Selection and Justification

Layered and independent of the transport. A single boundary separates moving bytes from everything else, which gives a deterministic simulator, unit tests without sockets and a direct answer to the missing radio: a Bluetooth transport would be one more implementation, not a rewrite. **One `asyncio` thread instead of several threads.** Four activities share three structures; on one loop they cannot race, so correctness does not depend on locking discipline. **Peer-to-peer instead of client–server.** A server would be a single point of failure and exactly the infrastructure that fails in a disaster. **Link state instead of flooding or on-demand discovery.** Flooding breaks down under load, and discovering a route on demand delays the first emergency packet. **SQLite per node in WAL mode.** It is a file with no server, and the API can read while the engine writes.

4.4 Data Flow Diagram (DFD)

Figure 2 is the context diagram (one node, no server); Figure 3 splits the node into six processes and five data stores.

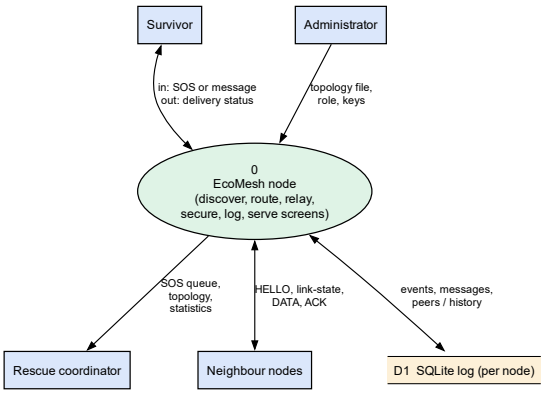


Figure 2: Level-0 (context) data flow diagram

4.5 UML Diagrams

The six UML diagrams of Phase II match the implemented design. **Use case diagram** (Figure 4): five actors; raising an SOS includes sending a message, and holding and flushing extends sending when there is no route. **Class diagram** (Figure 5). `Node` owns the discovery, gossip, routing, relay, ACK-tracking, retention and storage objects (filled diamonds); the router and the gossip share one `MeshGraph`; the relay owns its priority queue and uses a duplicate set; three classes implement `Transport`. **Activity diagram** (Figure 6). An arriving packet is validated and checked for duplicates. If it is addressed to this node, its signature is verified and its payload

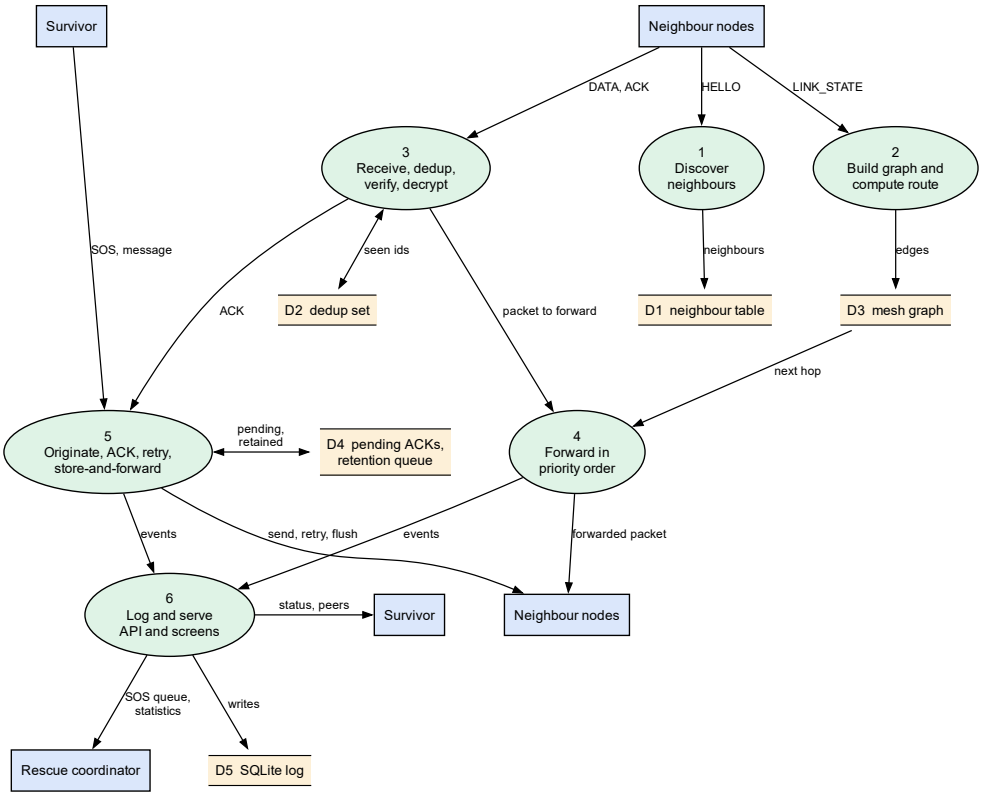


Figure 3: Level-1 data flow diagram of one node

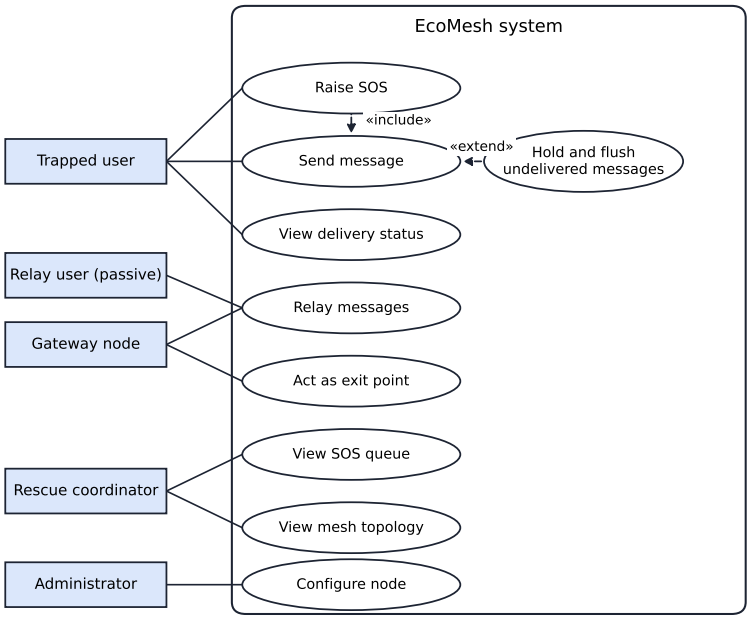


Figure 4: Use case diagram

decrypted, and it is logged, acknowledged and delivered; otherwise the TTL, loop and route checks decide whether it is dropped or queued again by priority. **Sequence diagram** (Figure 7). L1 creates, encrypts and

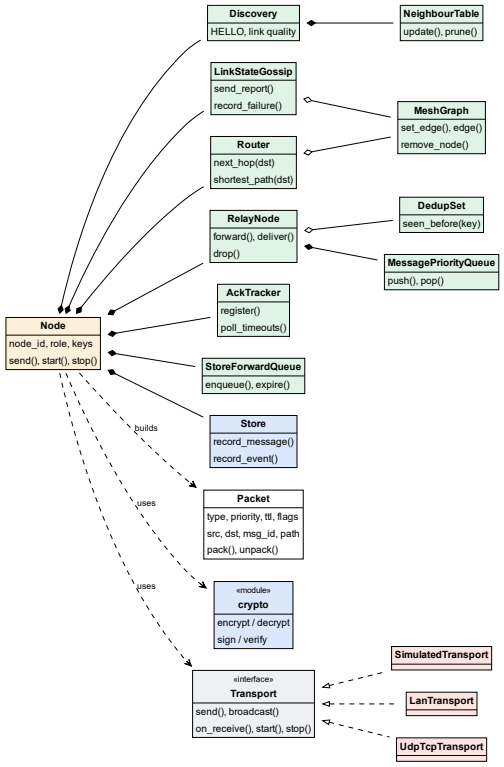


Figure 5: Class diagram: `Node` as the composition root

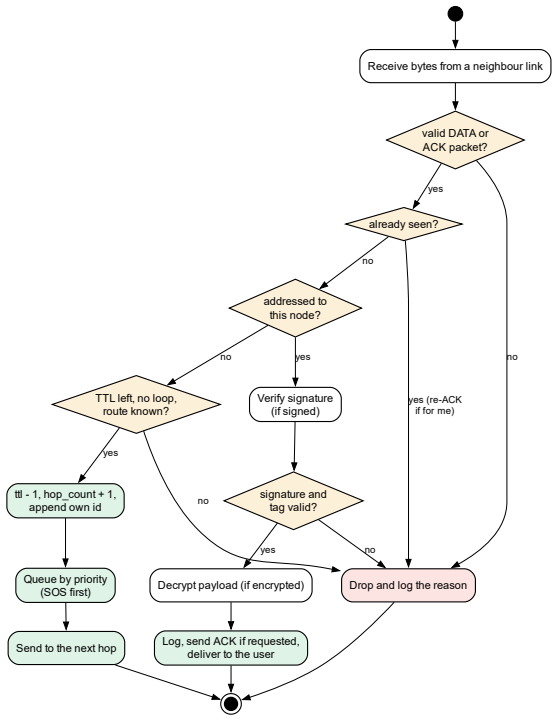


Figure 6: Activity diagram: the packet receive path

signs an SOS; each relay checks duplicates, lowers the TTL and adds its id; L5 delivers it and the ACK returns. **State machine** (Figure 8, left). A message is created, becomes pending (sent and awaiting an ACK) or held (no route), and ends delivered or failed; these states are the `status` values stored in the database. **Deployment diagram** (Figure 8, right). Five node processes, each with its own engine, web screen and SQLite file, connected only by the TCP links that the topology file lists; there is no server (Section 7.7).

4.6 Database Design

Every node keeps its own SQLite database in WAL mode, accessed with plain `sqlite3` and SQL; with only three tables an ORM would add indirection and nothing else. Every metric is a query over this database (SRS 6.1). **Relational schema.** `messages`(`msg_id` PK, `src`, `dst`, `priority`, `direction` in sent/received/forwarded, `status` in pending/delivered/failed/queued_sf, `body`, `created_ms`, `resolved_ms`, `hop_count`, `path`); `events`(`id` PK, `ts_ms`, `kind`, `msg_id`, `peer`, `detail`); `peers`(`node_id` PK, `role`, `last_seen_ms`, `link_quality`, `public_key`). **Keys and integrity.** The UUID `msg_id` joins messages to events; `body` holds plaintext only at source and destination; upserts move a message from pending to queued_sf to delivered in one row; `events.msg_id` and `events.peer` are logical foreign keys, since a relay logs events for messages it never stores. **Normalisation.** Third normal form, except the comma-separated `path`, always read whole. **Indexing.** `idx_events_ts` serves the live panel and `idx_msg_status` the held-message count. **Sample records** are in Table 6; the relay's row has no body.

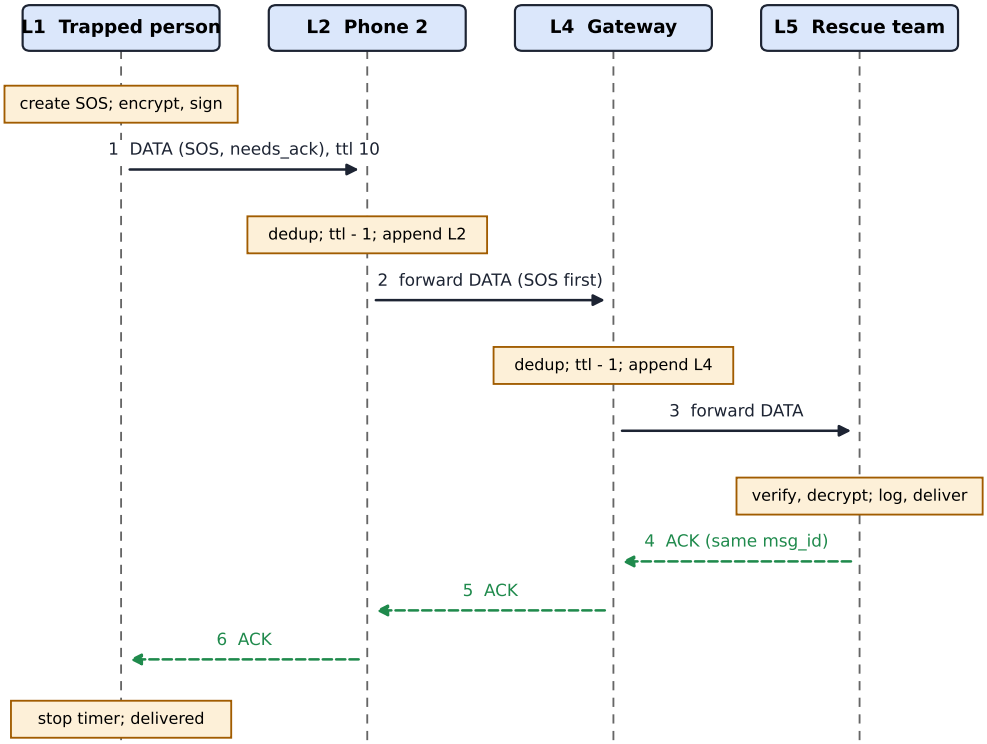


Figure 7: Sequence diagram: an SOS relayed hop by hop with the ACK returning along the reverse path

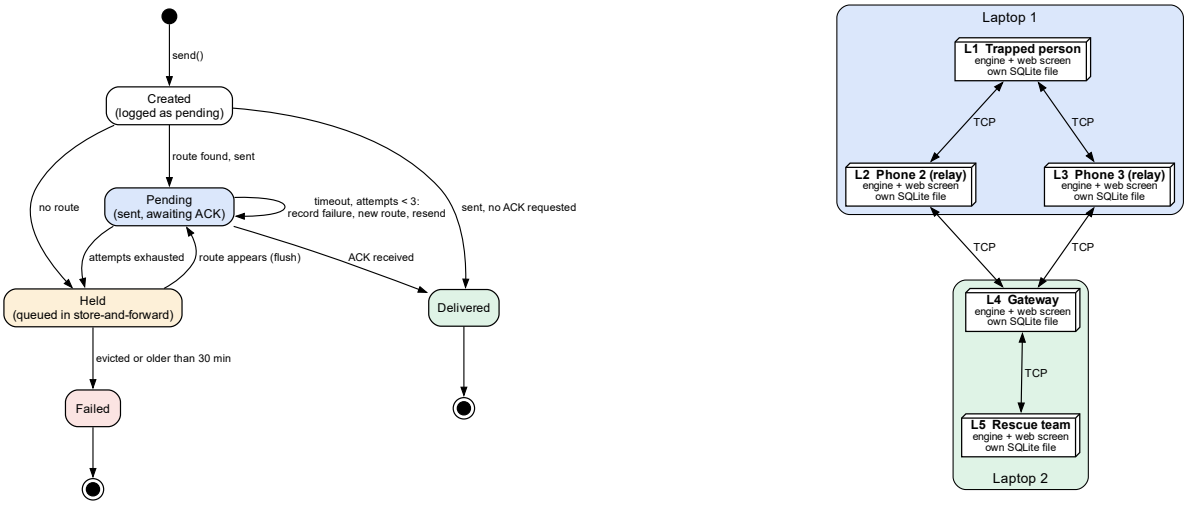


Figure 8: State machine of the message lifecycle (left) and deployment diagram of the live configuration (right)

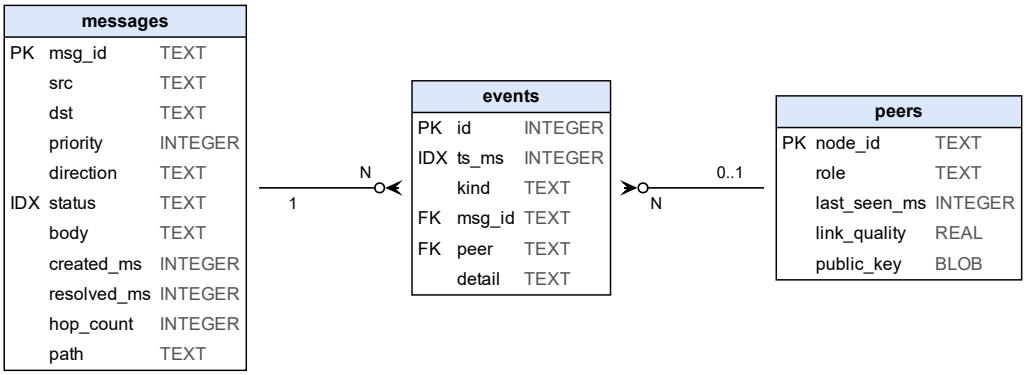


Figure 9: Entity–relationship diagram of the per-node database (PK primary key, IDX secondary index, FK logical foreign key)

Table 6: Sample rows of `messages` for one SOS from A to C in a real run: each node holds its own view of the same message id

DB	msg_id	src	dst	prio	direction	status	body	hops	path
A	8e859198...	A	C	0	sent	delivered	trapped on floor 3	1	B
B	8e859198...	A	C	0	forwarded	delivered	NULL	1	B
C	8e859198...	A	C	0	received	delivered	trapped on floor 3	1	B

4.7 Security and Error Handling Design

Table 7 links each threat to its countermeasure and requirement. **Authentication.** A node's identity is its

Table 7: Threat model and countermeasures

Threat	Mechanism	Requirement
Relay reads a message	AES-256-GCM payload; relays see only routing fields	SEC-1, 2, FWD-7
Relay modifies a message	GCM tag fails at the destination; dropped, logged	SEC-5
Relay forges an SOS	Ed25519 signature over the identity fields	SEC-3, 4, 6
Replay of a captured packet	dedup set keyed on message id (300 s)	REL-3
Malformed or hostile bytes	strict validation; `PacketError`; drop	FWD-6
Endless circulation	TTL per hop; own id in the path is a loop	FWD-3, 5
Hand-rolled cryptography	only `cryptography`-library primitives	SEC-7

Ed25519 key pair, announced in HELLO and gossip. **Authorisation.** Only GATEWAY and RESCUE nodes accept `RESCUE__` traffic; any node may relay but never decrypts or stores what it does not own. One pre-shared key covers the mesh, so a node holding it could read what it forwards; per-pair keys would prevent this (Section 9.11). **Input validation and exceptions.** Every field is range-checked; a violation raises `PacketError` and the packet is dropped and logged, never raised into the event loop; `verify` returns `False` instead of raising. **Logging.** Each drop has its own event (`drop_ttl`, `drop_loop`, ...). **Sensitive data.** Fresh 12-byte nonces; plaintext only at the two ends; private keys never sent; the signature skips fields that change in transit (TTL, hops, path).

CHAPTER 5

Algorithm Design and Data Structures

5.1 Identification of Critical Operations

Ten operations decide correctness and performance: (1) neighbour discovery and link quality; (2) link-state dissemination; (3) least-cost routing; (4) the per-packet forwarding decision; (5) priority scheduling; (6) duplicate suppression; (7) send, acknowledge and retry; (8) bounded store-and-forward; (9) encryption and signing; (10) resolving `RESCUE__`. They are Algorithms 1–10 of the Phase II *Design of Algorithms*, shown here as implemented.

5.2 Algorithm Design

A1 Discovery and link quality.

```
every 3 s: broadcast HELLO(role, public_key)                        // ttl = 1
on HELLO from p:  new <- table.update(p);  if new: on_peer_up(p)
    q <- hellos_received / hellos_expected   in a 20-slot window    // in [0,1]
    if |q - last_reported| >= 0.1: report q                         // avoids recomputing
    on noise
every 1 s: for p silent > 10 s: remove p; on_peer_down(p)
```

A new neighbour is scored over the HELLOs *due* so far, otherwise every fresh link would look lossy.

A2 Link-state with re-flooding.

```
every 5 s: seq++; send report(origin=me, seq, [(p, q, failures, role, key) for p in
    neighbours]) to each neighbour
on report(origin, s, entries) from sender:
    if origin = me or s <= seq_seen[origin]: return                  // stale or own: stop
    seq_seen[origin] <- s;  replace origin's outgoing edges with entries;  learn roles and
    keys
    if changed: invalidate route cache
    forward report unchanged to every neighbour except origin and sender   // controlled re
    -flooding
```

Without re-flooding a node knows only its neighbours' neighbours; sequence numbers admit each report once per link, so flooding terminates.

A3 Dijkstra and the cost function.

```
cost(u->v) = 1 + 5 * (1 - q) + 3 * f        // f = ACK timeouts on the link in the last 60
    s
dist[src] <- 0; heap <- {(0, src)}
while heap: (d,u) <- pop_min; if u visited: continue; mark u
    for (v, e) in graph.neighbours(u): if d + cost(e) < dist[v]: dist[v] <- d + cost(e);
    prev[v] <- u; push
next_hop(dst) = path[1] of the cached shortest path, or none (=> store-and-forward)
```

Derivation of the coefficients. The base 1 is the per-hop penalty; it keeps every weight positive (Dijkstra's condition) and prefers fewer hops between equal links. The term $5(1-q)$ makes loss expensive: a link delivering 40 % of its HELLOs costs $1+5(0.6)=4.0$, more than two clean hops (2.0) or three (3.0), which is exactly where minimum-hop routing errs. The term $3f$ is steeper than the estimated-loss term because an ACK timeout is *observed* failure: one adds the cost of three extra clean hops, and it applies at once so the retry avoids the failing link.

A4 Packet handling.

```
on packet p from link sender s:
    if not parse_ok(p) or p.type not in {DATA, ACK}: drop
    for_me <- p.dst = me or (p.dst = RESCUE__ and my role in {GATEWAY, RESCUE})
    key <- (type, msg_id) if for_me else (type, msg_id, attempt)     // a retry crosses
    relays that saw the original
    if dedup.seen_before(key): drop(duplicate); re-ACK if for_me; return
    if for_me: verify signature; decrypt; log; ACK if needs_ack; deliver
    elif ttl <= 0: drop(ttl_expired)   elif me in path: drop(loop)   elif no next_hop: drop
    (no_route)
    else: ttl--; hop_count++; path.append(me); queue.push(p, priority)            // A5
```

The key includes the *type*, because an ACK carries the id of the DATA it acknowledges (a bug found in integration), and at relays the *attempt*, so a retransmission is forwarded; the destination ignores the attempt and sees each message once. Only the originator retains (A8).

A5 Priority scheduling. A binary heap of (priority, timestamp, counter) pops the highest priority, then the oldest; applied at *every* hop, an SOS overtakes routine traffic again and again. **A6 Dedup.** A hash map of keys to first-seen time, pruned after 300 s. **A7 Send, ACK, retry.** A timed-out packet with fewer than 3 attempts gets a failure recorded on its link (cost +3), a recomputed route and a resend; after 3 attempts it moves to store-and-forward (Figure 10). **A8 Bounded store-and-forward.** `enqueue` evicts, when over 500 messages, the one with the largest (priority value, age), i.e. the lowest priority and oldest; `expire` drops anything older than 1800 s regardless of priority; the flush (every 10 s) sends each message whose destination now has a route. **A9 Security.** Send encrypts with a random nonce and signs the identity bytes; receive verifies against the sender's known key, then decrypts; either failure drops the packet. **A10 `RESCUE__`.** One Dijkstra run, then the nearest reachable node whose role is GATEWAY or RESCUE; none reachable means store-and-forward (Figure 11).

5.3 Flowcharts

Figure 10 follows a message from creation to acknowledgement, retry or retention, and Figure 11 the choice of next hop; the receive path is Figure 6.

5.4 Data Structure Selection

Table 8 lists each structure, its use and the reason.

Table 8: Data structures and why they were chosen

Structure and use	Reason
Dictionary of dictionaries: `MeshGraph` adjacency	$O(1)$ edge update; sparse, so no matrix
Binary min-heap: Dijkstra frontier, message priority queue	$O(\log n)$ insert and extract-min
Hash map with timestamps: dedup set, neighbours, route cache	$O(1)$ lookup; expiry by pruning
Deque of 20 slots per neighbour	bounded window for link quality
Dictionary by message id: ACK tracker, retention queue	$O(1)$ acknowledge and removal
Set: Dijkstra visited nodes, reachable set	$O(1)$ membership

5.5 Algorithm Selection and Justification

Flooding delivers, but its message count grows with the square of the node count and swamps a loaded mesh. Bellman–Ford ($O(VE)$) and A* (which needs a geometric heuristic) add nothing here; Dijkstra with a binary heap is correct for non-negative weights in $O((V + E) \log V)$. A heap gives $O(\log n)$ priority order and a hash set $O(1)$ duplicate checks.

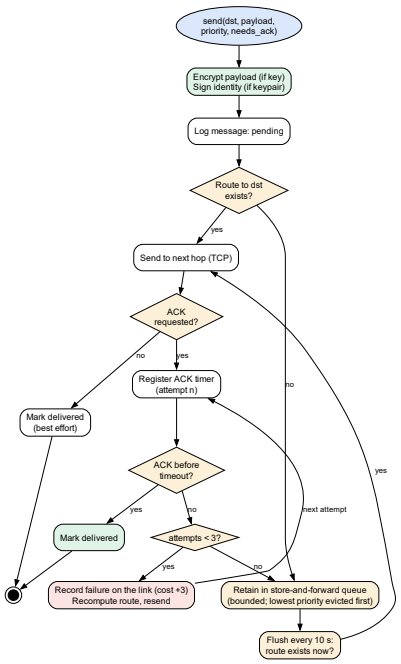


Figure 10: Flowchart: originate, acknowledge, retry and retain a message

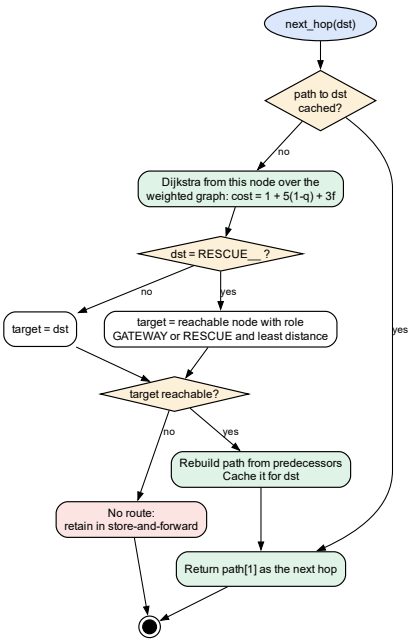


Figure 11: Flowchart: next-hop selection, including `RESCUE__`

5.6 Time Complexity Analysis

Table 9 gives the time complexity of the critical operations.

Table 9: Time complexity (V nodes, E links, n queued packets, q retained messages, D distinct destinations, $|N|$ neighbours)

Operation	**Time**
Dijkstra with the weighted cost	$O((V+E)\log V)$ in every case
Link-state merge on receiving a report	$O(\deg(v))$ plus $O(\|N\|)$ to re-flood
HELLO handling	$O(1)$ amortised
Priority enqueue / dequeue	$O(\log n)$; draining n packets $O(n \log n)$
Duplicate detection	$O(1)$ average
Loop check	$O(\|\text{path}\|)$, at most 10 here
Store-and-forward flush	$O(D\,(V+E)\log V + q)$ with the route cache; worst case $O(q\,(V+E)\log V)$
Eviction	$O(q)$ scan, only when over the bound
Packet parse and validate	$O(1)$ for the fixed 52-byte header

5.7 Space Complexity Analysis

The graph needs $O(V+E)$, Dijkstra $O(V)$ extra, the dedup set grows with traffic over 300 s, the retention queue holds at most 500 packets, and only the SQLite log on disk is unbounded.

5.8 Optimization Opportunities

Per-destination cache invalidation, multipoint relays [5], short retention at relays, batched commits and retuned timers (Chapter 8).

CHAPTER 6

Subject-Wise Engineering Concepts and APNI Leap Alignment

6.1 Concepts Applied, Subject by Subject

Software Engineering. Seven layers live in separate modules joined only in `node.py`; the message lifecycle is an explicit state machine (Figure 8); every requirement has an identifier and acceptance tests (Table 4); bad input is dropped and logged, never raised; Git with phase branches and Conventional Commits (Section 7.8). **Computer Networks.** Each node is both host and router. The custom binary protocol has a magic value, version and self-describing length, so a stream is framed without a prefix (Table 5); HELLO is a one-hop broadcast and DATA and ACK use TCP per hop with the previous hop's identity and 1.5 s connect bounds. Errors are recovered by TCP, by ACK, retry and re-routing, and by store-and-forward. A central server is deliberately absent; each peer is a server to its neighbours and a client of them. **DBMS.** Schema, keys, a deliberately denormalised `path`, two indexes, upserts and WAL (Section 4.6); every write commits at once; all metrics (delivery ratio, hop histogram, latency percentiles, retries, held and flushed counts) are SQL queries. Each node owns its file, so there is no shared database. **Operating Systems.** One process per node; five processes communicate only over TCP and HTTP. A single cooperative `asyncio` thread needs no locks. Scheduling is *priority scheduling* (SOS, NORMAL, STATUS, oldest first on ties) at every hop; Round-Robin was rejected because an emergency must not share a link fairly. A Windows cancellation race in `wait_for` was fixed with a cancellation-safe helper (D5). **Data Structures and Algorithms.** Adjacency dictionaries, binary heaps, hash maps and sets, and fixed-size deques (Table 8); Dijkstra is correct because every edge weight is at least 1 and runs in $O((V + E) \log V)$; the cost coefficients are derived (Section 5.2) and tested against hop count (Chapter 8). **Cloud Computing.** Not applicable by design: there is no server, hosted database or autoscaling, and no recurring cost.

6.2 Mapping of Subjects to Project Implementation

Table 10 maps each subject to the concept applied, its module and the evidence.

Table 10: Subjects, concepts applied, modules and evidence

Subject	Concept applied	Project module	Evidence
Software Engineering	modularity, state machine, error policy, traceability, version control	all; `node.py`	Table 4; Section 7.8
Computer Networks	binary protocol, framing, per-hop TCP, timeouts, failure detection	`packet.py`, `lan_transport.py`	`test_packet`, `test_lan_transport`
DBMS	schema, keys, indexes, upserts, WAL, query metrics	`store.py`, `api/metrics.py`	`test_store`; Section 4.6
Operating Systems	process per node, event loop, priority scheduling, bounded memory	`node.py`, `reliability.py`	concurrency tests; priority scenario
Data Structures	heap, hash map and set, adjacency dictionary, deque	`routing.py`, `reliability.py`	`test_reliability`, `test_dijkstra`
Algorithms, Graphs	Dijkstra, cost derivation, link-state flooding	`routing.py`	weighted vs hop count
Cloud Computing	not applicable (no server by design)	–	–

CHAPTER 7

Implementation and Module-Wise Coding

7.1 Development Environment

The system is written in Python 3.10 with `asyncio`, the `cryptography` library and SQLite; the API uses FastAPI, uvicorn and pydantic; the web application uses React 18, Vite and Tailwind; testing uses pytest and pytest-asyncio; version control is Git on GitHub. The source is about 6 500 lines of Python (`mesh/`, `api/`, `sim/`, `story/`), 4 500 lines of tests and 3 100 lines of React.

7.2 Overall Implementation Strategy

The build order followed what blocks what. **Phase 1:** the 52-byte packet and the `Transport` interface, so every track could work against a stable boundary. **Phase 2:** discovery, single-hop relay and the simulated transport, so that routing and reliability could be tested in one process. **Phase 3:** routing, reliability, security and storage, wired into `Node`. **Phase 4:** the API and React application, the scenario runner and metrics, engine fixes found by integration, and the multi-device deployment.

7.3 Module-Wise Implementation

Table 11 gives, for each module, its code reference (file and size), its purpose, input and output, processing logic, dependencies and test evidence.

7.4 Front-End Implementation

There is one app, the **device screen** (`story/node.html`): a single page that every device serves and that acts as a *survivor phone*, a *gateway* view or a *rescue console* depending on the device's role. A phone is only a screen; the engine runs on a laptop and the phone opens the page over the same Wi-Fi or hotspot. The survivor screen is designed to be read under stress. It has a plain status line; nine request types in a grid (Medical emergency, selected by default, Injured, Trapped, Fire or smoke, Unsafe building, Need water, Need food, Need medicine, and Elderly or child needs help); a location box; an SOS button that must be held for about 1.2 s so that it is not sent by accident; an "I am safe" check-in that never delays an SOS; a form for a text message to a named device; the rescue team's reply above a step-by-step progress tracker; and the nearby phones with their signal strength. **Voice input.** A *Speak* button beside the location box and the message box uses the browser's speech recognition to turn speech into text, which helps a person who is injured or cannot type. The recognition service in Chrome needs an internet connection and a page opened on `localhost` or `https`, so where it is missing the button is disabled with the reason, a failure such as a blocked microphone is explained in one line, and typing always works. The rescue console lists each request with its type, location, route and age, keeps check-ins in a separate Marked safe list, filters by kind and by request type, offers replies with quick phrases, a message history filtered by priority and direction, CSV export, counters, and a network map that shows link quality and highlights the route of each new message. Every screen also has a live panel, "What this device is doing", that turns the device's event log into plain English. Screens are shown in Section 10.2. The **simulation lab** (`ui/`, a React PWA served by `api/server.py`) is not part of the demonstration; it remains because it produced the experiments of Chapter 8.

7.5 Back-End Implementation

The back end is FastAPI with a WebSocket. The engine server (`api/server.py`) serves the simulation lab: snapshots, kill and revive, link edits, routes (weighted and hop-count), statistics, a message's full story, a protocol inspector, a security drill, scenario jobs, and `/ws`, which streams every logged event. Each device server (`story/node.py`) exposes `/api/me` (everything its screen needs), `/api/sos` (the need and location travel as structured fields), `/api/send` (a text message to a named device), `/api/safe` (an "I am safe" check-in at the

Table 11: Modules: code reference, logic, dependencies and tests

Module (lines)	Purpose, input → output, processing, details	Depends on	Tests
`mesh/core/packet.py` (362)	wire format, `Packet` ↔ bytes. `struct` format asserted to be 52 bytes; every field range-checked; `unpack` rejects bad magic and truncation of each region; `remaining_length` frames stream reads; `identity_bytes` is the signed content	–	47 + 4 000 fuzz
transports (461)	bytes between neighbours: simulated (seeded loss, latency), multicast/TCP, and the multi-device `LanTransport` (Section 7.7)	`packet`	15 + 6
`discovery.py` (292)	HELLOs → neighbour table, peer events, link quality over a 20-slot window reported only when it moves by 0.1; `visible_neighbours` range emulation	`transport`	15 + 12
`relay.py` (153)	received packets → deliver, forward or drop (algorithm A4); one drain task over a priority queue	`reliability`	18 + 3
`routing.py` (546)	reports → graph, next hop. `MeshGraph`, `edge_cost`, sequence-numbered gossip with quality, failures, role and key, cached Dijkstra, `RESCUE__` terminals, hop-count switch for the comparison	`discovery`	14 + 14 + 33
`reliability.py` (240)	`DedupSet` (300 s), `MessagePriorityQueue`, `AckTracker` (3 attempts), `StoreForwardQueue` (500 messages, 1800 s)	`packet`	18 + 20
`crypto.py` (131)	AES-256-GCM with nonce prefix and associated data; Ed25519 sign and non-raising verify; one exception type	`cryptography`	19
`store.py` (258)	tables, WAL, upserts, event queries, `events_since`, listing and counting for metrics	`sqlite3`	23
`node.py` (513)	composition root: `send()` encrypts, signs, logs, dispatches; receive verifies, decrypts, logs, ACKs; ACK loop records the failure, recomputes, resends; flush loop expires and retries	all above	27
`sim/`, `api/lab.py`, `api/metrics.py` (3 000)	simulator with a virtual clock and seven scenarios; metrics read from node logs	`node`	5 + 12

lowest priority), `/api/events` (the event log in plain English), `/api/history` (messages filtered by priority and direction), `/api/export.csv` (the received requests, rescue device only) and `/api/power`; started with `--phone` it serves its screen on every network interface so a phone on the same Wi-Fi can open it. Request bodies are validated with pydantic, and engine errors map to clear HTTP errors.

7.6 Database Implementation

The three tables of Section 4.6 are created with `CREATE TABLE IF NOT EXISTS`, WAL mode is set at connection time, and the two indexes are created with the schema. Queries used include retained messages by status (`SELECT * FROM messages WHERE status = ?`), a message's timeline (`SELECT * FROM events WHERE msg_id = ? ORDER BY ts_ms`), event counters (`SELECT COUNT(*) FROM events WHERE kind = ?`), and the upsert `INSERT ...ON CONFLICT(msg_id) DO UPDATE`. Every write commits immediately.

7.7 Communication and Concurrency Implementation

The original `UdpTcpTransport` could not carry a complete `Node`: it never learned a HELLO sender's address, and it reported the packet's *original* source instead of the previous hop, which breaks link-state flooding. `lan_transport.py` fixes both. A JSON **topology file** lists every device (role, host, port) and the links (who

can hear whom); a device talks only to its configured neighbours, which emulates limited radio range as stated in the scope and satisfies DIS-7, so no multicast is needed. Each connection starts with the previous hop's identity followed by the unmodified packet. A **power switch** makes a device neither send nor hear, like a dead radio. `python -m story.node --id L1` starts one device (engine plus a small web server); five of them, started together on one machine or one per laptop, form the mesh, each with its own SQLite log, port and screen. In the two-laptop demonstration L1–L3 ran on one laptop and L4–L5 on the other, joined through a phone hotspot (Figure 8). Recovery is layered: TCP per hop; ACK, retry and re-route end to end; store-and-forward when retries run out.

7.8 Git Repository Setup

Repository: `https://github.com/tanvisheth1234-wq/miniproject`. **Structure:** `mesh/core/` (engine), `api/`, `sim/`, `story/`, `ui/`, `tests/`, `docs/`, `config/`, with README, CONTRIBUTING and requirements from the first phase. **Branching strategy:** one branch per phase, each inheriting the previous and never merged back (`develop` to `develop_7`; the last holds the final screens and documents). **Commit conventions:** Conventional Commits (`feat(routing): ...`, `fix(reliability): ...`). **Contribution of members:** Miti Shah built discovery, the relay, the node skeleton and the simulated transport in Phase 2 and, in Phase 3, Dijkstra pathfinding with gateway and rescue handling and route caching, the dedup set and priority queue, ACK tracking with retry and the store-and-forward queue, AES-256-GCM encryption with Ed25519 signing, and the SQLite log; Twissha Shah built the weighted cost function and link-state gossip, the wiring of routing, reliability, crypto and storage into the node, the multi-device program and device screens, the robustness and socket tests, and the evidence scripts; Tanvi Sheth built the packet format, transports, the API, the React application and the scenario runner, the Phase 4 engine fixes found by integration, and the nine request types and voice input of the final device screen. Nine of Miti's Phase 3 commits, made from Twissha's laptop, were re-attributed to Miti on 24 October 2026 with everyone's agreement (`docs/contribution-log.md`). **Evidence of development history:** Table 12.

Table 12: Commits per team member, read from `git log` over all remote branches (98 commits)

Member	Commits	Share
Miti Shah	31	32 %
Twissha Shah	35	36 %
Tanvi Sheth	32	33 %

Conventional Commits. 92 of 98 commits (94 %) follow the `type(scope): subject` convention of `CONTRIBUTING.md`.

Branches: `main`, `develop`, `develop_2`, `develop_3`, `develop_4`, `develop_5`, `develop_6`, `develop_7`. **Tags:** `v0.3-phase3`.

7.9 Integration of Modules

`Node` registers one dispatcher on the transport that feeds each packet to discovery (HELLO), gossip (LINK_STATE) and relay (DATA, ACK); each ignores types it does not own. Callbacks join the pieces: peer-up adds a graph edge and logs it, a topology change invalidates cached routes, relay hooks write the log, and the ACK loop reports failures to gossip at once. The screens read the same objects and database as the engine.

7.10 Implementation Challenges and Resolutions

The main difficulties (Table 18) were a receive callback heard by only one layer, dead nodes that stayed routable, signatures that failed after the first relay (fixed by signing identity fields only), ACKs lost at relays, a hostile packet raising `UnicodeDecodeError`, a Windows hang in `Node.stop()`, and a transport unable to run a full node (fixed by `LanTransport`).

CHAPTER 8

Scalability, Performance and Resource Management

Every number here is generated from stored measurements by `docs/report/evidence/`. The *simulator* (real nodes over an in-memory transport with seeded loss on a virtual clock) and *live devices* (real processes, TCP and timers) are never mixed.

8.1 Scalability Requirements

The SRS requires 50 nodes, 90 % delivery on 40 nodes at 20 % loss, convergence within 15 s and recovery within 10 s (Section 1.10); sizes above five exist only in the simulator.

8.2 Workload and Load Model

Each scenario fixes a topology, per-link loss, 5–40 ms latency, 10–90 messages and a failure script, repeated over seeds so routing modes see identical traffic. Sweeps vary loss 0–50 % (21 nodes) and size 10–65 nodes; throughput uses 100–2 000 sends; the live run sent 30 SOS with 8 relay failures and 4 gateway outages.

8.3 Resource Utilization

Simulator: 65 nodes used about 13 MB of heap (Table 14). **Live:** about 55 MB and 0.2–1.0 % of a core per idle process (Table 15). **Storage:** one log row per event; at most 500 held messages. **Network:** transmissions per delivery in Table 14.

8.4 Concurrent Processing

A node is single-threaded and cooperative with no locks. On a three-node line with ACKs, 7029 messages per second were delivered at 1 000 simultaneous sends and 4686 per second at 2000 (all ACKed within 1.0 s); the concurrency tests (Chapter 9) confirm exactly-once delivery.

8.5 Scheduling and Synchronization Analysis

Scheduling is by priority, oldest first on ties, at every hop; its metric is the SOS latency advantage under congestion (Section 8.7), since an idle network has nothing to overtake. No synchronisation primitive is needed: shared state lives on the event-loop thread.

8.6 Performance Measurement

One Windows 11 laptop, Python 3.10; live runs used five loopback processes with shortened timers. Memory and CPU come from OS counters, live latency from polling every 20 ms, and simulator metrics from SQL over the node logs.

8.7 Performance Results

Reliability under loss (Figure 12a, 21-node grid, 8 seeded runs per point). Delivery stays at or above 98 % to 20 % loss and falls to 61 % at 50 %; each lost ACK costs a 4 s timeout, so median latency goes from tens of milliseconds at 0 % to seconds at 20–30 %.

Scenarios. The 40-node collapse scenario (20 % loss, a relay on the active path killed at $t = 10$ s, the gateway killed at 25 s and revived at 60 s) delivered 92.6 % on average over 50 seeded runs (worst 79 %, best 100 %). At 35 % loss (outside the specified condition) delivery was 89.1 %. **Weighted versus hop-count routing (Figure 12b).** A 2-hop shortcut crosses a link with 45 % loss while a 3-hop path is clean, and

Table 13: Results of the seven simulator scenarios (weighted routing; delivery ratio shown as mean (min–max) over the runs)

Scenario	Nodes	Loss	Runs	Delivery	Hops	Conv. (s)	Recov. (s)	S&F flushed / queued
baseline_10	10	5%	20	100.0 % (100–100)	1	5.5	–	7/7
collapse_40	40	20%	50	92.6 % (79–100)	4	9.6	8.1	2919/3117
no_gateway	12	5%	20	100.0 % (100–100)	2	5.2	–	207/207
high_loss	20	35%	20	89.3 % (74–100)	2	17.7	–	1045/1076
chain_10hop	11	5%	20	100.0 % (100–100)	9	14.4	–	87/90
hopcount_vs_ weighted	5	45%	20	99.2 % (90–100)	2	5.2	–	–
priority_load	16	0%	20	100.0 % (100–100)	2	7.6	–	1356/1356

Hops = median hop count of delivered messages. Conv. = mean seconds until every node's graph contains every node. Recov. = mean seconds from relay death until the sender has a route avoiding it. S&F counts are event counts summed over all runs: a message re-queued after a failed retry is counted each time, and messages still held when a run ends are not flushed.

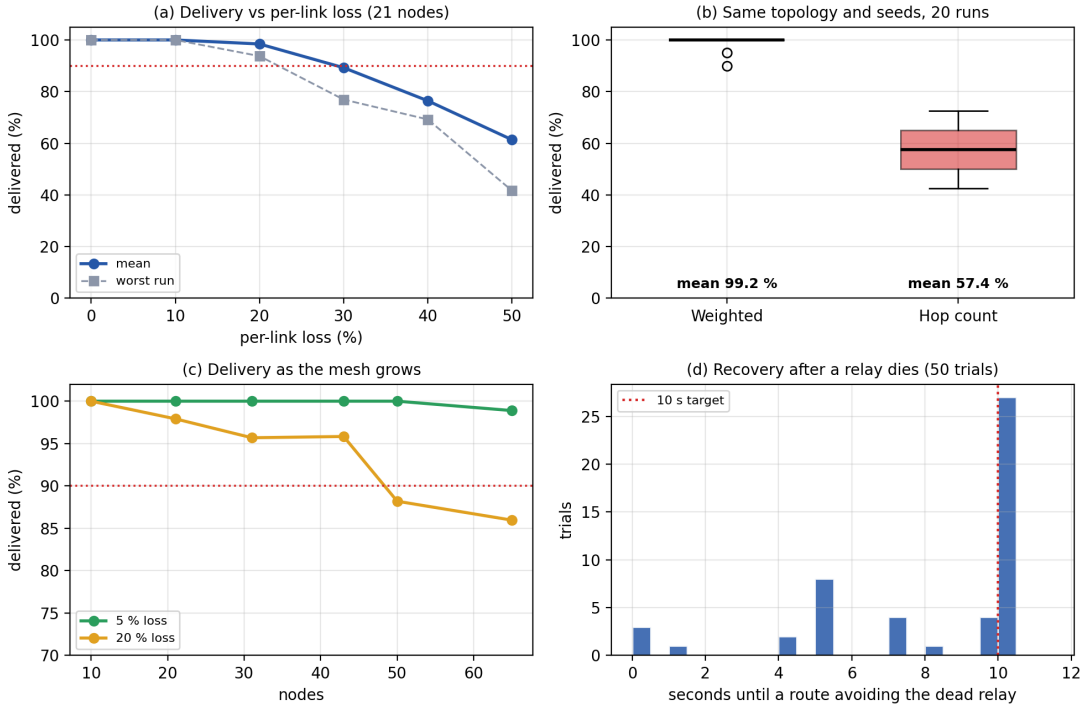


Figure 12: (a) Delivery against per-link loss. (b) Weighted versus hop-count routing on identical seeds. (c) Delivery as the mesh grows. (d) Recovery time after a relay death, 50 trials.

messages are best-effort, so a poor route shows directly as loss. Weighted routing delivered 99.2 % and hop count 57.4 %, weighted winning on 20 of 20 identical seeds. The topology is *designed* to expose hop count, so it shows that the cost avoids a lossy shortcut, not that every mesh has this gap.

Priority under load. The congested scenario (16 nodes, 90 messages to the same destination, 15 % SOS) gives an SOS a median latency 4.4× lower than routine traffic. Only here do both take the same trip, so the comparison is fair.

Scalability. At 5 % loss delivery is 100 % to 50 nodes and 98.0 % at 65; at 20 % loss it is 92.9 % at 50 and 83.1 % at 65. Transmissions per delivery grow about linearly with size, not quadratically as with flooding.

Recovery and convergence follow the timers (Figure 12d): convergence comes in multiples of the 5 s

Table 14: Scalability, 10 to 65 simulated nodes (10 runs per size and loss rate)

Nodes	Links	Delivery at 5% loss	Delivery at 20% loss	Tx per delivered (5%)	Tx per delivered (20%)	Conv. max at 20% (s)	CPU (s, 5%)	Heap (MB, 5%)
10	13	100.0 %	100.0 %	5.0	8.9	15.2	3	0.7
21	32	100.0 %	98.8 %	7.4	16.1	15.2	13	1.7
31	50	100.0 %	96.2 %	10.6	26.5	15.2	43	3.0
43	72	100.0 %	96.2 %	12.2	24.8	20.2	91	5.3
50	85	100.0 %	92.9 %	14.2	31.9	15.2	110	6.5
65	113	98.0 %	83.1 %	20.6	39.4	15.2	188	9.8

Nodes and links include the gateway and rescue post added to the grid. Tx = hop-by-hop data transmissions. CPU and heap are for the whole simulated mesh in one process on a virtual clock, not the cost of one real device.

gossip interval, and recovery took under 10 s in 24 of 50 trials and 10.2–10.25 s in the rest, bounded by the 10 s heartbeat timeout.

Live devices. Table 15 gives five real processes over real TCP.

Table 15: Live devices: 5 operating-system processes, real TCP sockets, real timers, on one laptop (loopback)

Measurement	Result
Process start → every device knows every device	4.4 s
SOS arrives at the rescue device (30/30 delivered)	median 32 ms, 95th pct 63 ms, max 78 ms
ACK back at the sender	median 47 ms, 95th pct 87 ms
Relay switched off → sender re-routes (8 trials)	median 4.1 s, max 4.4 s; every later SOS delivered
Gateway down, 5 SOS held; gateway back (4 trials)	100% flushed, median 4.2 s, max 4.2 s
Device API response (200 requests)	median 16 ms, 95th pct 31 ms
Per-process memory and CPU, idle	55 MB; 0.2–1.0% of one core
Per-process CPU while sending SOS (includes the harness polling every 20 ms)	up to 12% of one core

Timer settings: HELLO 1 s, neighbour timeout 4 s, link-state 2 s, ACK timeout 3 s. Latency is measured by polling each device's own API every 20 ms (up to 20 ms of error). Wireless delay and loss are not included.

8.8 Bottleneck Identification

The 10 s heartbeat timeout (which bounds recovery), the 4 s ACK timeout under loss, the 5 s link-state interval, one SQLite commit per event and whole-cache route invalidation.

8.9 Optimization and Improvement

No optimisation followed these measurements, so there is no before-and-after comparison; the data point to a shorter, adaptive neighbour timeout, an immediate link-state report on change, an adaptive ACK timeout and per-destination invalidation.

8.10 Cloud Deployment and Resource Utilization

Not applicable: there is no server and no cloud deployment.

8.11 Scalability Limitations

Tested to 65 simulated nodes and 5 live devices; larger meshes, wireless interference and mobility were not tested.

CHAPTER 9
Testing and Validation

9.1 Testing Strategy

Testing is layered (unit, integration, functional, non-functional, boundary, concurrency, live devices); the simulated transport makes tests deterministic, so a failure reproduces from its seed. The 334 test cases were run completely 3 times. Complete suite runs: 3; fully green: 3. No inconsistent tests.

9.2 Unit Testing

Each module is tested alone: the packet format (47 tests), cost function and graph, Dijkstra with caching and `RESCUE__`, gossip, discovery, the dedup set, priority queue, ACK tracker, store-and-forward bounds, AES-GCM, Ed25519, the SQLite log, relay and simulated transport.

9.3 Integration Testing

`test_node` (28 tests) covers multi-hop sending, ACK, the message lifecycle, end-to-end encryption and signing, retry exhaustion, eviction and expiry; `test_ack_multihop` checks ACKs and retries across relays; `test_api` and `test_device_api` drive the servers and engine together; `test_lan_transport` runs a three-node mesh over real TCP.

9.4 Functional Testing

Table 4 summarises how the requirements are verified. 68 functional requirements: 52 verified by automated tests alone, 12 by automated tests plus a manual check of the screen, 0 partially implemented, 2 by code inspection, 2 by manual check only. All mapped tests passed in every complete run.

The device screens are also tested as a user meets them: `device_walkthrough.cjs` starts five devices, drives the survivor phone, a second phone, the gateway and the rescue console in a real browser, and records each check (21 of 21 pass, Table 16). DT-19 to DT-21 cover the final screen: exactly nine request types with Medical emergency as default, a new type reaching the console and its filter, and Speak buttons that fill the boxes from a stand-in recogniser, explain a network failure and are disabled without browser support. `ui_walkthrough.cjs` checks the simulation lab (16 of 16). Both scripts are run by hand, outside `pytest`.

9.5 Non-Functional Testing

Reliability: delivery at 0–50 % loss (Figure 12a). **Scalability:** 10 to 65 nodes (Table 14). **Performance:** latency by priority (Section 8.7), throughput and live latency (Table 15). **Availability:** 50 simulated recoveries and 8 live failovers. **Security:** relays never see plaintext; tampering, wrong keys, forged signatures and unknown signers are rejected. **Usability:** one 1.2 s hold, plain-language status, optional voice input; no study with participants.

9.6 Boundary and Invalid-Input Testing

Empty, header-only and truncated input; bad magic; out-of-range TTL and lengths; over-long paths and ids; wrong-length signatures and keys; ciphertext shorter than the nonce; a send to an unknown destination (held, not raised). `test_robustness` feeds 3 000 random and 1 000 bit-flipped packets to the parser and 200 garbage buffers to a running node, which must reject them and keep working.

9.7 Concurrency and Stress Testing

200 simultaneous sends over two hops arrived exactly once with every ACK; 50 SOS and 50 routine messages from two senders arrived intact; a packet replayed five times was delivered once; throughput to 2 000 sends and

Table 16: Browser checks of the device screens (the one app), run against five live device processes

ID	Check	Result
DT-01	Survivor screen loads and the app bar names the device	Pass
DT-02	Rescue device shows the console, with its name in the app bar	Pass
DT-03	The mesh organises itself: the survivor is connected to the rescue team through phones in between	Pass
DT-04	Releasing the SOS button after 0.4 s sends nothing	Pass
DT-05	A held SOS with need and location reaches the rescue console with the route it took	Pass
DT-06	The survivor screen confirms "The rescue team has your request"	Pass
DT-07	"I am safe" from another phone appears under Marked safe, not among the requests	Pass
DT-08	A typed message sent to the rescue team is delivered and listed on the console	Pass
DT-09	Reply opens and a quick reply "Help is on the way" reaches the survivor	Pass
DT-10	A typed reply survives the 0.7 s refresh (text and focus kept) and reaches the survivor	Pass
DT-11	Console filters: SOS only, Messages only, and by need	Pass
DT-12	History filter shows only the chosen priority; the CSV button is shown and the export has a header and rows	Pass
DT-13	The "What this device is doing" panel lists live events in plain language	Pass
DT-14	The gateway view says it is online and shows the messages it passed on	Pass
DT-15	With the gateway off, an SOS shows "Saved, waiting for a path"	Pass
DT-16	With the gateway back on, the held SOS is delivered and appears on the console	Pass
DT-17	At 390 px and 320 px wide neither screen scrolls sideways	Pass
DT-18	No script errors or console errors on any device screen during the run	Pass
DT-19	The request screen offers the nine SOS options, no "Need help", and "Medical emergency" is selected by default	Pass
DT-20	A new option ("Unsafe building") reaches the rescue console with its location and appears in the need filter	Pass
DT-21	Voice to text: the Speak buttons fill the location and message boxes, explain a network error, and are disabled where the browser has no speech recognition	Pass

the congested priority scenario complete the stress tests.

9.8 Test Case Specification

Table 17 specifies eight representative test cases.

Table 17: Representative test cases

ID	Req.	Scenario, input and preconditions	Expected result	Actual result and evidence	Result
TC-01	RTE-3, 4	short path over a lossy link, long path over clean links	Dijkstra takes the clean path	as expected; `test_dijkstra`	Pass
TC-02	PRI-3	SOS reaches a relay with routine packets queued	SOS forwarded first	as expected; `test_relay`	Pass
TC-03	REL-5, 6	ACK lost; sender times out	route recomputed, resent, destination sees it once	as expected; `test_ack_multihop`	Pass
TC-04	SNF-1, 2	no route; a route later appears	held, then flushed and delivered	as expected; live: 4 of 4 outages flushed (Table 15)	Pass
TC-05	FWD-6	3 000 random byte strings	only `PacketError`; node survives	one defect found and fixed (D4); `test_robustness`	Pass
TC-06	K1	40 nodes, 20 % loss, relay and gateway failures, 50 seeds	delivery $\geq$ 90 %	92.6 % (worst run 79 %); Table 13	Pass
TC-07	K3	kill the relay on the active path, 50 trials	re-route $\leq$ 10 s	mean 8.1 s; 24 of 50 within 10 s; worst 10.25 s; Figure 12	Partial
TC-08	K4, K6	congested radio; weighted versus hop count on identical seeds	SOS $\geq$ 3$\times$ faster; weighted delivers more	4.4$\times$; 99.2 % vs 57.4 %; Section 8.7	Pass

9.9 Defect Identification and Resolution

Table 18 records each significant defect, its cause, fix and retest; D4 and D5 were fixed before the Chapter 8 measurements.

Table 18: Significant defects, causes, corrective actions and retesting

ID	Found by	Defect and cause	Fix and retest
D1	integration test	one receive callback: only the last of discovery and relay heard packets	single dispatcher; regression test
D2	Phase 4 integration	dedup keyed on message id dropped every ACK at the first relay and swallowed retries	key on type, id and attempt; retry counter in the reserved byte; new tests
D3	Phase 4 integration	dead nodes stayed routable: newer reports merged with old ones	a newer report replaces the origin's edges; tests
D4	fuzz test	a non-ASCII node id made `Packet.unpack` raise `UnicodeDecodeError`, which receivers do not catch	decode failures raise `PacketError`; fuzz tests pass
D5	repeated socket-test runs	`Node.stop()` hung in about a quarter of runs on Windows: `wait_for` swallowed a cancellation	helper on `asyncio.wait`; 15/15 and 40/40 runs clean
D6	repeated full-suite runs	two timing-sensitive tests failed in 2 and 1 of 6 runs under load	closed: a `wait_until` polling helper replaces the fixed sleeps
D7	repeated full-suite runs	history-order test failed now and then: same-millisecond messages had no defined order	closed: ties broken by row id; 30/30 runs pass under load
D10	final review run	attack-drill test failed once in 15 runs: fixed sleeps, destination had not yet learned the sender's key	closed: the drill polls for its conditions; 30/30 pass

9.10 Validation Against KPIs

Table 19 compares each target of Table 1 with the measured result. K2 and K3 hold on average but miss by

Table 19: Validation of the key performance indicators of Table 1 against measured data

KPI	Target	Measured	Verdict
K1 Delivery	$\geq$ 90 % (40 nodes, 20 % loss, failures injected)	92.6 % mean over 50 runs (worst run 79 %). Without failures, a 43-node grid at 20 % loss: 96.2 %.	Met
K2 Convergence	$\leq$ 15 s	collapse_40: mean 9.6 s; 47 of 50 runs $\leq$ 15 s; worst 15.25 s. Mean 17.7 s in the 35 % loss scenario. Values come in multiples of the 5 s gossip interval. Live devices (2 s gossip): 4.4 s from process start.	Met in 47/50 runs; not at 35 % loss
K3 Recovery	$\leq$ 10 s	mean 8.1 s, median 10.2 s; 24 of 50 trials $\leq$ 10 s; the other 26 took 10.2–10.25 s, i.e. the 10 s heartbeat timeout plus the 0.25 s sampling step. Live devices (4 s timeout): median 4.1 s, max 4.4 s over 8 trials, every message delivered	Mean met; 26/50 trials miss by $\leq$ 0.25 s
K4 SOS advantage	$\geq 3\times$	4.4× (median routine / median SOS)	Met
K5 Store-and-forward	100 % flushed	207/207 in the no-gateway scenario; 100 % of live trials	Met
K6 Weighted vs hop count	Weighted better	99.2 % vs 57.4 %	Met
K7 Scale	$\geq$ 50 nodes within the targets	50 nodes at 5 % loss: 100.0 % delivery; 50 nodes at 20 % loss: 92.9 %; largest tested 65 nodes: 98.0 % (5 %) and 83.1 % (20 %)	Met
K8 UI freshness	$\leq$ 2 s	API response median 16 ms; screens poll every 0.7 s	Met
K9 Correctness	100 % passed	334/334 passed in all 3 runs	Met
K10 Security	All hold	relay cannot decrypt, forged signature rejected, tampered payload rejected: automated tests pass	Met

under a second in some runs (timer constants); K7 holds to 50 nodes but not at 65; all others are met.

9.11 Limitations and Untested Scenarios

No radio hardware (interference, fading, mobility untested); live timings on loopback only; no usability study; voice input checked with a stand-in recogniser, not real speech; browser checks run by hand; nothing above 65 nodes; relays do not retain; one pre-shared key and no external security review.

CHAPTER 10

Results, Demonstration and Discussion

10.1 Implemented Features

Table 20 lists what was built and what was not, with evidence.

Table 20: Feature status

Feature	Status	Evidence
Engine: discovery, gossip, weighted Dijkstra, priority, TTL, dedup, ACK and retry, store and forward, AES-GCM, Ed25519, SQLite	Implemented	334 tests; Chapters 8 and 9
Simulator, metrics and simulation lab (API, website)	Implemented	7 scenarios to 65 nodes; 16 of 16 browser checks
Device app: survivor, gateway and rescue screens; SOS, "I am safe", messages, replies, filters, history, CSV, live panel, network map	Implemented	device-API tests; 21 of 21 browser checks (Table 16)
Multi-device deployment over real sockets	Implemented	5 processes measured; two-laptop demonstration
Nine request types; voice input for location and message	Implemented	device-API tests; DT-19 to DT-21
Flush on a new neighbour (SNF-3); filtered history (LOG-5)	Implemented	automated tests, browser checks
BLE or Wi-Fi Direct radio; per-pair keys; usability study	Not built	future work

10.2 Screenshots and Demonstration Evidence

All screenshots come from five running devices (`python -m story.local`) following `docs/DEMO_SCRIPT.md`: an SOS is delivered through a relay phone, the rescue team replies, the gateway is switched off and the next SOS is held, and the held SOS is delivered when the gateway returns. The same story also ran on separate laptops joined by a phone hotspot (Figure 8); that run was not timed.

10.3 Functional Results

Of the 68 requirements 52 are verified by automated tests, 12 with a screen check, 2 by inspection and 2 manually; none is partial. The key stories (multi-hop SOS, re-routing around a failed relay, hold and later delivery) ran on real devices, and the browser run repeats them with the nine request types and voice input (21 of 21 pass).

10.4 Performance Results

Table 19: 92.6 % delivery on 40 nodes at 20 % loss with failures, SOS 4.4× faster under load, live SOS in a median 32 ms, failover in about 4.1 s, held messages flushed in about 4.2 s.

10.5 Comparison with Existing Systems

Other systems were not run, so the comparison rests on documented properties (Table 3) and one fair baseline, hop count on identical seeds (99.2 % against 57.4 %); no delivery claim is made for systems that publish no measurements.

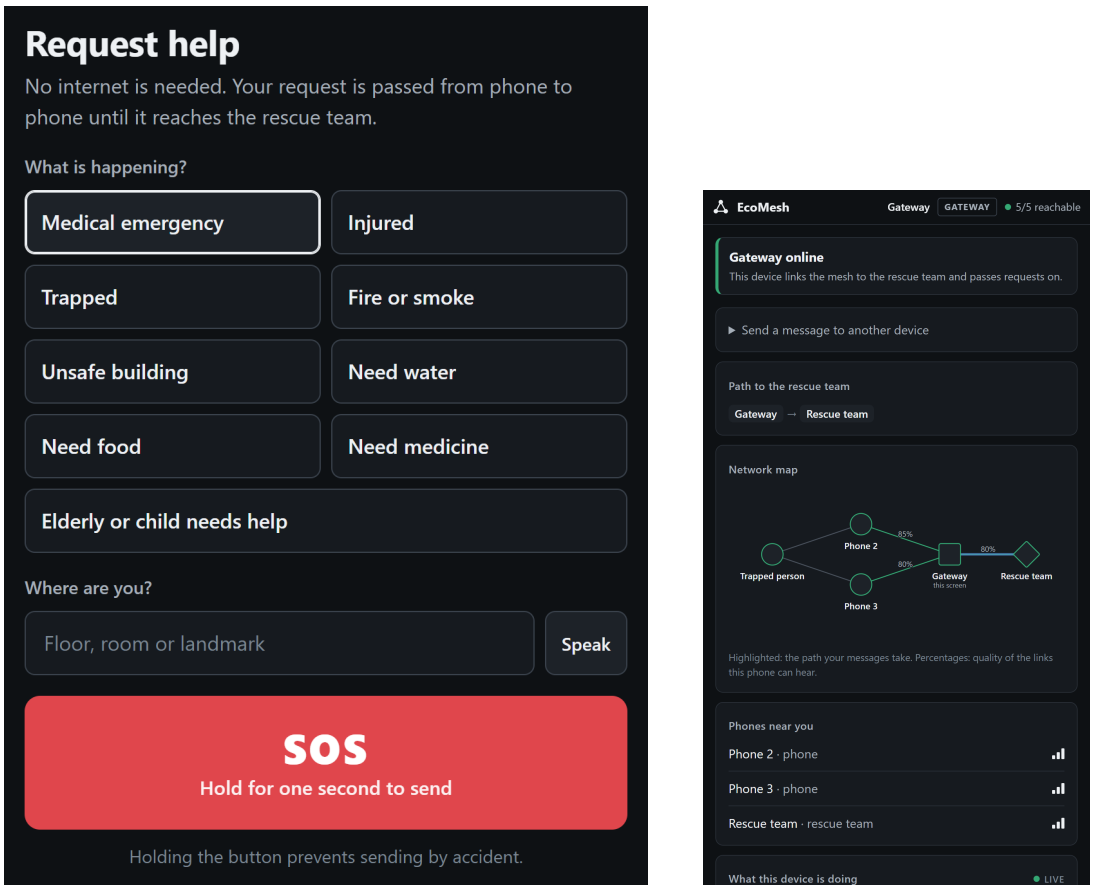


Figure 13: Left: the survivor's request form with nine request types (Medical emergency selected by default), the location box with its Speak button for voice input, and the SOS button that must be held. Right: the gateway's view, with the path to the rescue team, the network map and the nearby phones.

10.6 Achievement of Objectives

Table 21 maps each objective to its evidence.

Table 21: Objectives, supporting evidence and status

Obj.	Evidence	Status
O1	live convergence 4.4 s; simulator 5–15 s in multiples of the gossip interval	Achieved (47 of 50 runs $\leq$ 15 s)
O2	99.2 % vs 57.4 % on 20 identical seeds (Figure 12b)	Achieved
O3	4.4× lower median latency under load (Section 8.7)	Achieved
O4	92.6 % at 40 nodes and 20 % loss; recovery mean 8.1 s, 24 of 50 trials within 10 s	Partly achieved
O5	207 of 207 flushed in simulation; 4 of 4 live outages	Achieved
O6	security tests (Section 9.5)	Achieved with a pre-shared key
O7	every metric is a query over per-node logs	Achieved
O8	web application, device screens, API tests, five processes, two laptops	Achieved

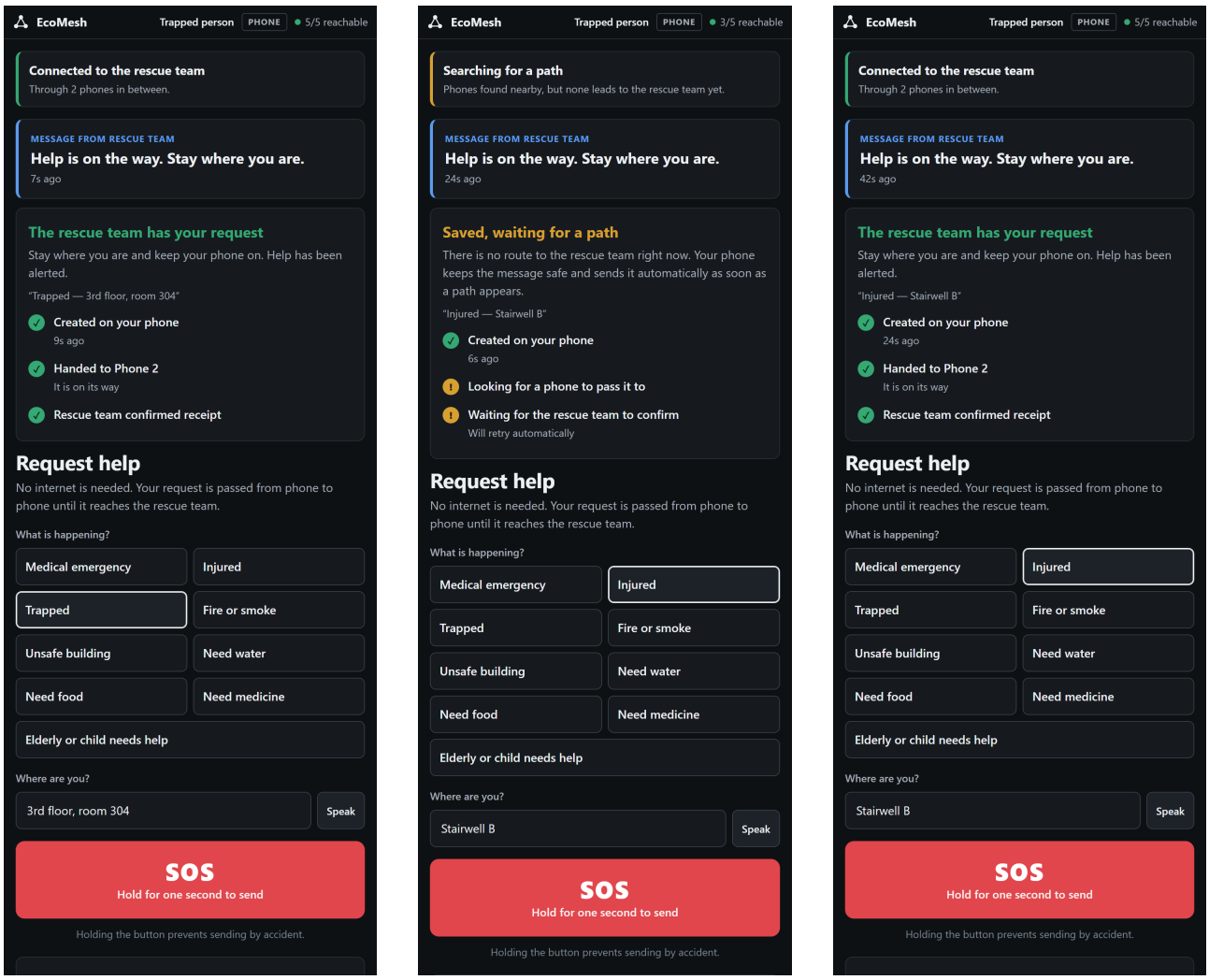


Figure 14: The survivor's screen. Left: a "Trapped" SOS is delivered through Phone 2 and the gateway, and the rescue team's reply appears above the progress tracker. Centre: the gateway is off, the status reads "Searching for a path" and the new "Injured" SOS is "Saved, waiting for a path". Right: the gateway is back and the held SOS has been delivered.

10.7 Discussion of Findings

Hop count is a poor proxy for delivery: a weak shortcut loses almost half the traffic under hop count and almost none under the weighted cost. Timer constants, not algorithms, set recovery and convergence, which shows what to tune. Priority matters once links queue, which is the disaster condition.

10.8 Practical Applicability

A deployment needs devices running the node, a topology file (or multicast), a gateway or rescue device and a shared network; it works today as a building safety system on laptops, and a radio transport would make it phone-to-phone.

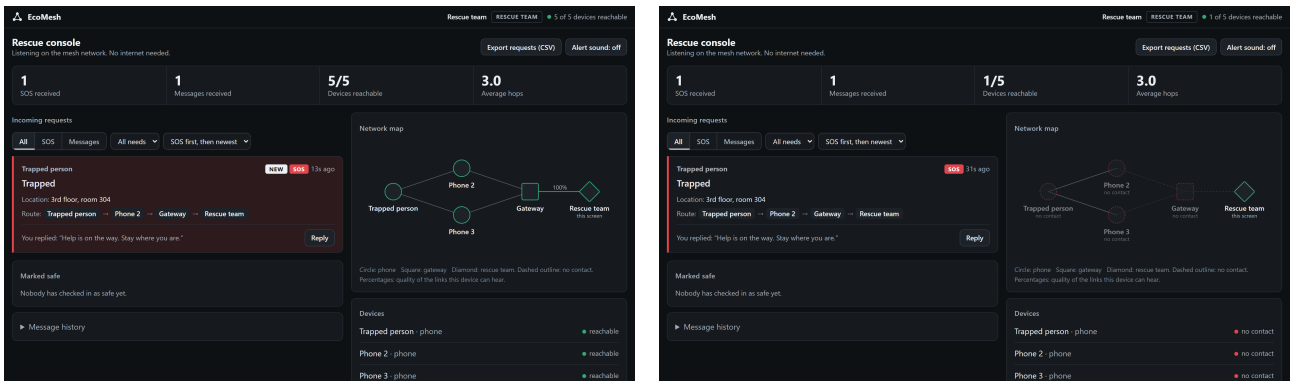


Figure 15: The rescue console. Left: the request with its type, location, age and route and the reply already sent, with counters, filters, the Marked safe list, history and a network map highlighting the route. Right: with the gateway off the rescue device can reach no one ("1 of 5 devices reachable") and earlier requests stay visible.

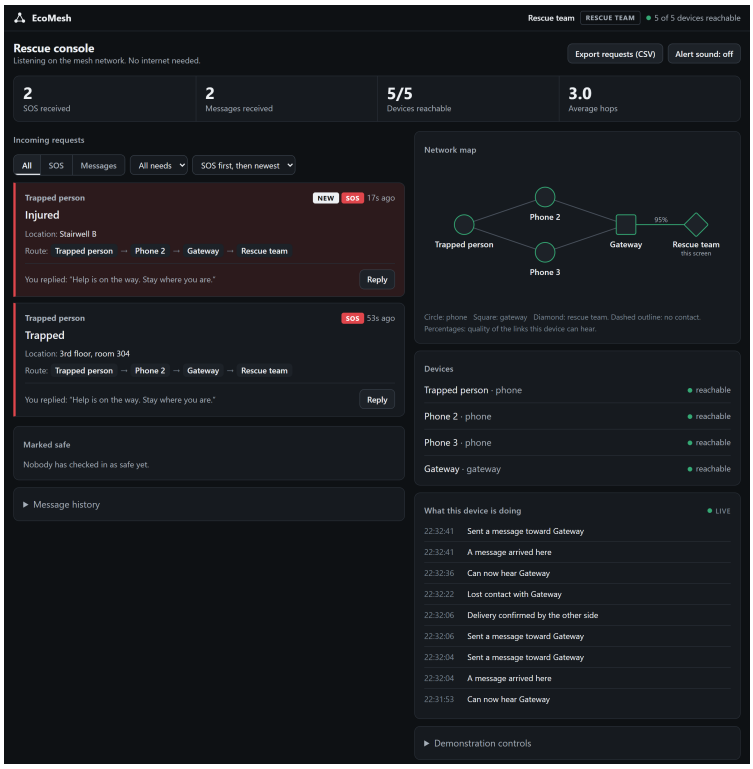


Figure 16: The rescue console after the gateway returned: the held "Injured" SOS has arrived, and the live panel "What this device is doing" (lower right) records the outage in plain English, from "Lost contact with Gateway" to "Can now hear Gateway" and "A message arrived here".

CHAPTER 11
Conclusion and Future Scope

11.1 Conclusion

Every mainstream messaging tool fails when its infrastructure fails, although devices in a stricken building can still reach each other. We built EcoMesh, a peer-to-peer mesh in which every device both sends and relays, with a complete engine, a simulator that measures it, a REST and WebSocket interface, one device app for survivors, gateways and rescuers (with nine request types and voice input), and a multi-device deployment over TCP. Weighted routing delivered 99.2 % of messages where hop-count routing delivered 57.4 % on identical seeds; an SOS had a 4.4× lower median latency under load; a 40-node mesh at 20 % loss with injected failures delivered 92.6 % over 50 runs; and on five live processes an SOS arrived in a median of 32 ms, the sender re-routed around a failed relay in about 4.1 s, and every message held during a gateway outage was delivered when the gateway returned.

11.2 Achievement of Objectives

Objectives O1–O3 and O5–O8 were achieved. O4 was achieved for delivery but only partly for recovery time, where 26 of 50 trials exceeded 10 s by at most 0.25 s, the heartbeat timeout (Table 21).

11.3 Major Engineering Contributions

A transport-agnostic engine that runs unchanged on a simulator and on real sockets; a derived link-quality cost function with an immediate failure penalty and network-wide re-flooding; per-hop priority with a bounded queue that never evicts an SOS for routine traffic; signatures that survive multi-hop relaying; and a measurement framework in which every number in this report is generated from per-node logs and stored results.

11.4 Current Limitations

As detailed in Section 9.11: a neighbour-restricted LAN stands in for a radio; one pre-shared key serves the mesh; relays do not retain; timer constants quantise recovery and convergence; there is no usability study with participants, and the browser checks are scripts run by hand rather than part of the automated suite; there is no phone application, only a web page that a phone opens over the same Wi-Fi; voice input depends on the browser's online speech service, so it is unavailable when the internet is down and typing remains the fallback; and scale was tested to 65 simulated nodes.

11.5 Future Enhancements

(1) A Bluetooth LE or Wi-Fi Direct transport and an Android client speaking the same protocol. (2) Adaptive timers: a shorter fixed neighbour timeout (about 7 s) was tried and dropped because it lowered delivery at 20 % loss, so the next step is a timeout adapted to each link, an immediate report on topology change and an ACK timeout derived from the measured round-trip time. (3) X25519 per-pair key agreement. (4) Short retention at relays, so that a relay does not lose a message it has accepted. (5) Per-destination route invalidation. (6) Voice input that works offline, using an on-device speech model instead of the browser's online service. (7) A usability study, the browser checks inside the automated suite, and tests beyond 65 nodes.

11.6 Lessons Learned

Build the simulator first and draw the transport boundary before the layers: it paid for itself in testing and in deployment. Integration finds what unit tests do not (the multi-hop ACK bug survived every single-node test). Look at distributions as well as means: recovery clustered at the heartbeat timeout, which the mean hid. Record who built what from the first week, with consistent Git identities.

References

[1] M. R. Albrecht, J. Blasco, R. B. Jensen and L. Mareňková, "Mesh Messaging in Large-scale Protests: Breaking Bridgefy," in *Topics in Cryptology – CT-RSA 2021*, Lecture Notes in Computer Science, vol. 12704, Springer, 2021, pp. 375–398.

[2] C. E. Perkins and P. Bhagwat, "Highly dynamic Destination-Sequenced Distance-Vector routing (DSDV) for mobile computers," in *Proc. ACM SIGCOMM*, 1994, pp. 234–244.

[3] D. Johnson, Y. Hu and D. Maltz, "The Dynamic Source Routing Protocol (DSR) for Mobile Ad Hoc Networks for IPv4," RFC 4728, IETF, Feb. 2007.

[4] C. Perkins, E. Belding-Royer and S. Das, "Ad hoc On-Demand Distance Vector (AODV) Routing," RFC 3561, IETF, July 2003.

[5] T. Clausen and P. Jacquet (eds.), "Optimized Link State Routing Protocol (OLSR)," RFC 3626, IETF, Oct. 2003.

[6] J. Moy, "OSPF Version 2," RFC 2328, IETF, Apr. 1998.

[7] J. Chroboczek and D. Schinazi, "The Babel Routing Protocol," RFC 8966, IETF, Jan. 2021.

[8] D. S. J. De Couto, D. Aguayo, J. Bicket and R. Morris, "A high-throughput path metric for multi-hop wireless routing," in *Proc. ACM MobiCom*, 2003, pp. 134–146.

[9] A. Vahdat and D. Becker, "Epidemic Routing for Partially-Connected Ad Hoc Networks," Technical Report CS-2000-06, Duke University, 2000.

[10] K. Fall, "A delay-tolerant network architecture for challenged internets," in *Proc. ACM SIGCOMM*, 2003, pp. 27–34.

[11] V. Cerf *et al.*, "Delay-Tolerant Networking Architecture," RFC 4838, IETF, Apr. 2007.

[12] D. G. Reina, M. Askalani, S. L. Toral, F. Barrero, E. Asimakopoulou and N. Bessis, "A survey on multihop ad hoc networks for disaster response scenarios," *International Journal of Distributed Sensor Networks*, vol. 11, no. 10, art. 647037, 2015.

[13] M. Dworkin, "Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC," NIST Special Publication 800-38D, Nov. 2007.

[14] D. J. Bernstein, N. Duif, T. Lange, P. Schwabe and B.-Y. Yang, "High-speed high-security signatures," *Journal of Cryptographic Engineering*, vol. 2, no. 2, pp. 77–89, 2012.

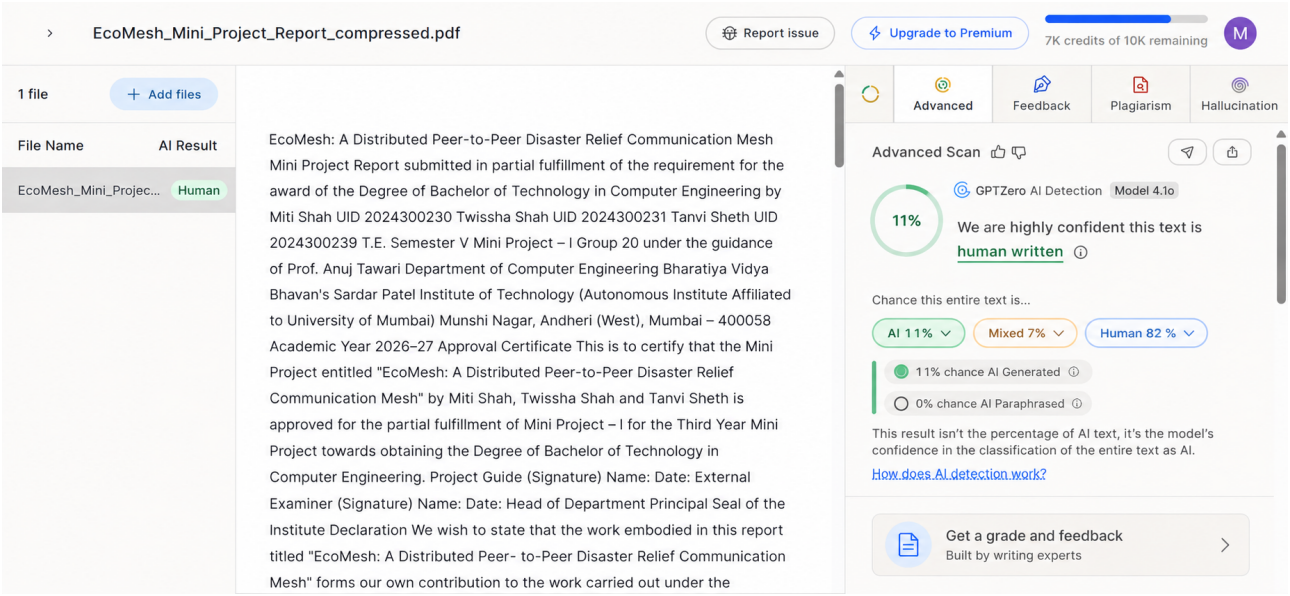
[15] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.

[16] The Briar Project, "Briar: secure messaging anywhere – how it works," `https://briarproject.org`, accessed 2026.

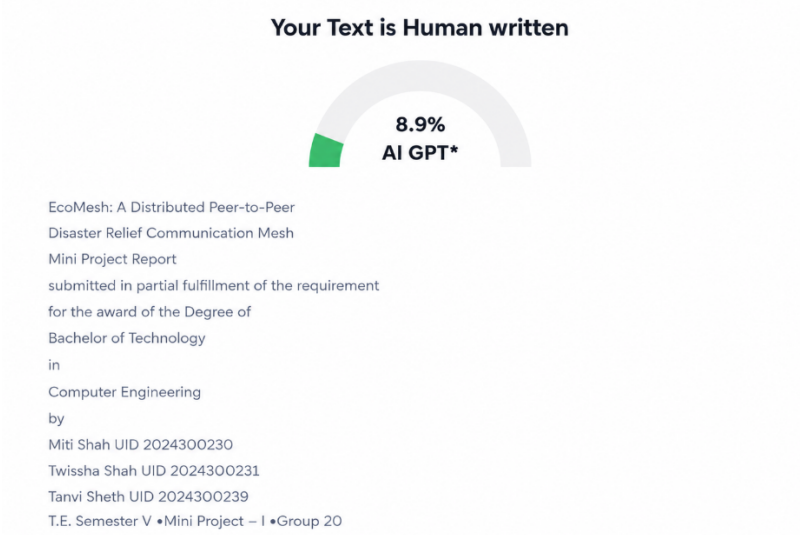
[17] Meshtastic, "Mesh Broadcast Algorithm" and "Why Meshtastic uses managed flood routing," documentation, `https://meshtastic.org/docs/overview/mesh-algo/`, accessed 2026.

[18] The Serval Project, "Serval: communications for disaster and remote areas" (including the Rhizome data-distribution protocol), project documentation, `https://www.servalproject.org`, accessed 2026.

Appendix: Research Paper / Poster and Plagiarism Report



GPTZero advanced scan (model 4.1o): 11 % AI, 7 % mixed, 82 % human; "highly confident this text is human written".



Second AI-text detector: 8.9 % AI; "Your text is human written".